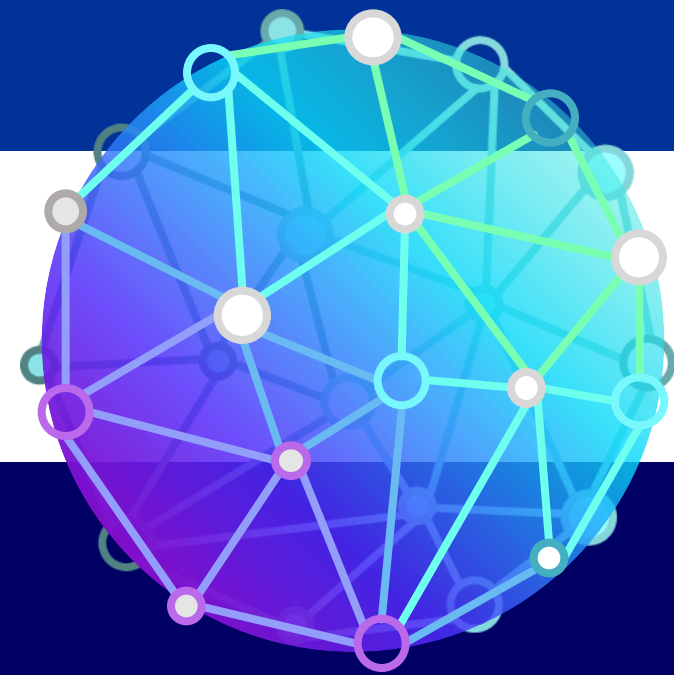




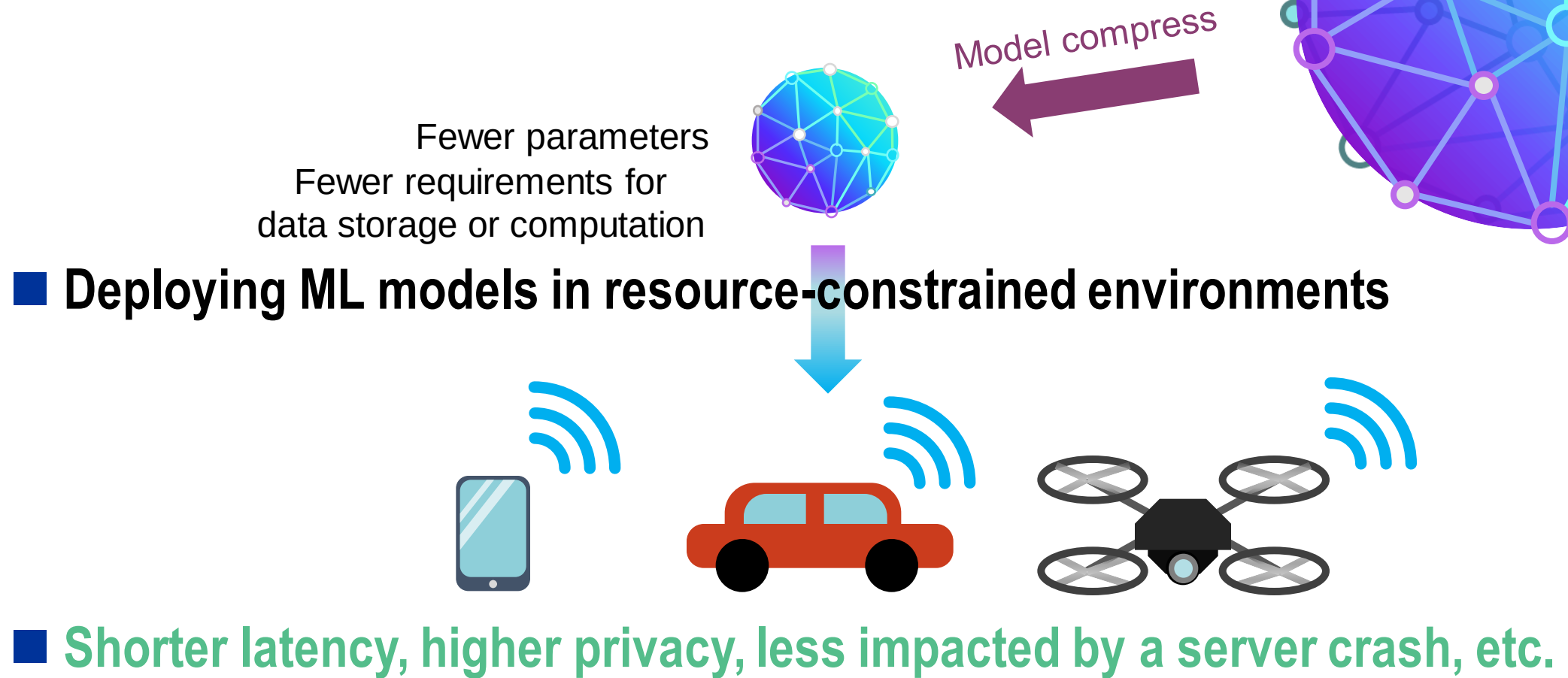
DNN Models Have High Accuracy

But they have **HUGE** model size

Cannot directly be deployed on edge devices! ☹️



We Need A Smaller Model!



Pruning and Sparsity

Ning-Chi Huang

黃霽琪



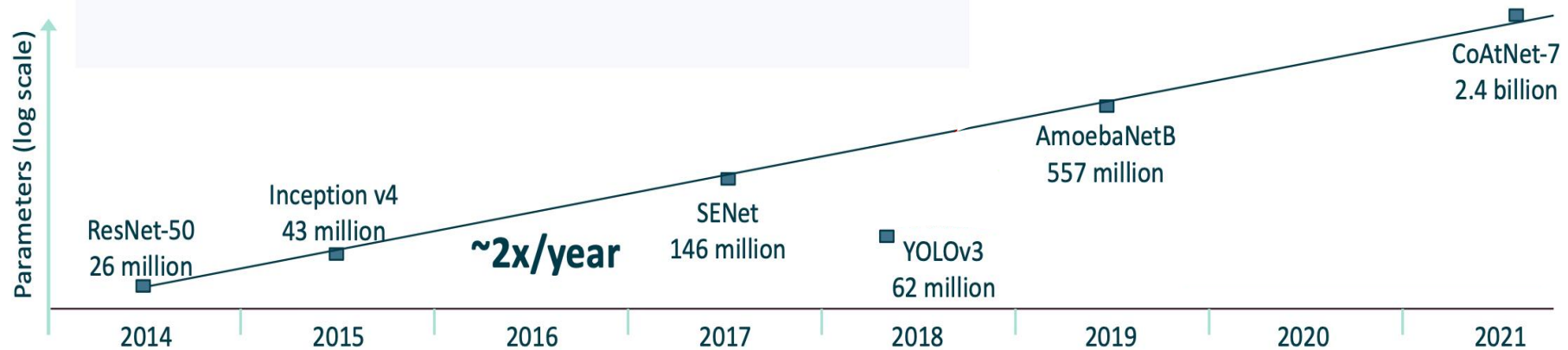
Outline

- Motivation of pruning
- Concept of pruning
- Introduction of different pruning methods
- Hardware for pruned/sparse NNs
- Fine-tuning/training pruned neural networks



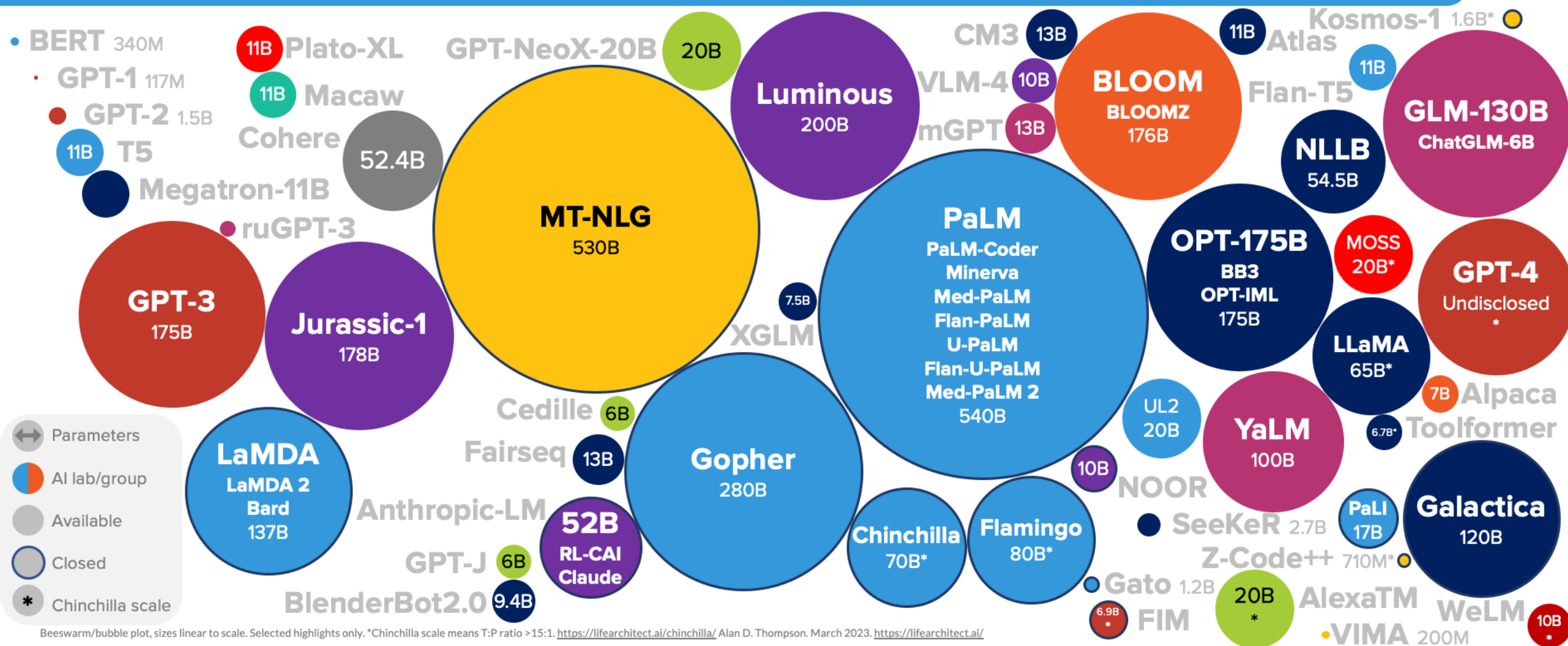
Trends in AI Development

Model size is increasing exponentially!



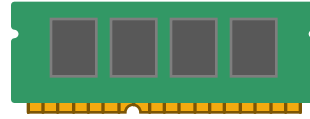
Source: https://community.cadence.com/cadence_blogs_8/b/breakfast-bytes/posts/linleyspr22

LANGUAGE MODEL SIZES TO MAR/2023

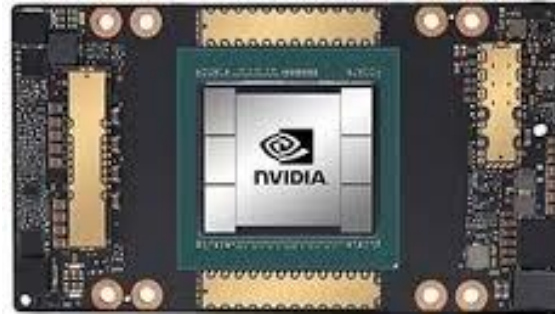


Everything is Expensive on Edge Devices

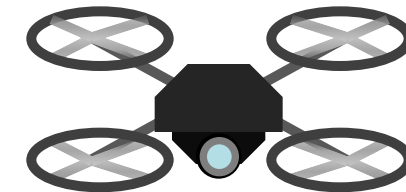
■ Memory resources



■ Computational resources



■ Energy consumption

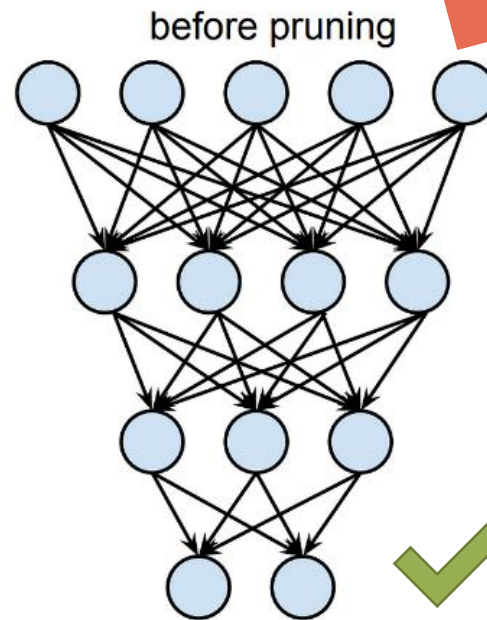


What is Network Pruning?

- Networks are typically over-parameterized

- A huge number of redundant weights!

- Prune them!



CAT ●



CAT ●



Outline

- Motivation of pruning
- Concept of pruning
- **Introduction of different pruning methods**
- Hardware for pruned/sparse NNs
- Fine-tuning/training pruned neural networks



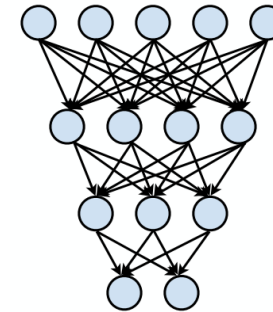
Which Weights or Neurons Can be Pruned?

- Importance of a synapse:

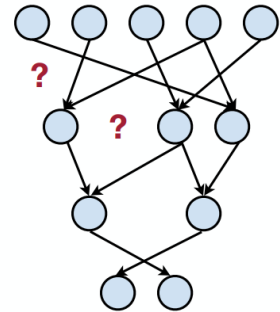
- Absolute values, life long, ...

- Importance of a neuron:

- The number of times it wasn't zero on a given data set ...



Pruning
.....➔



- After pruning, the accuracy will drop 😞

- Fine-tuning on training data for recovery
- Don't prune too much at once, or the network won't recover

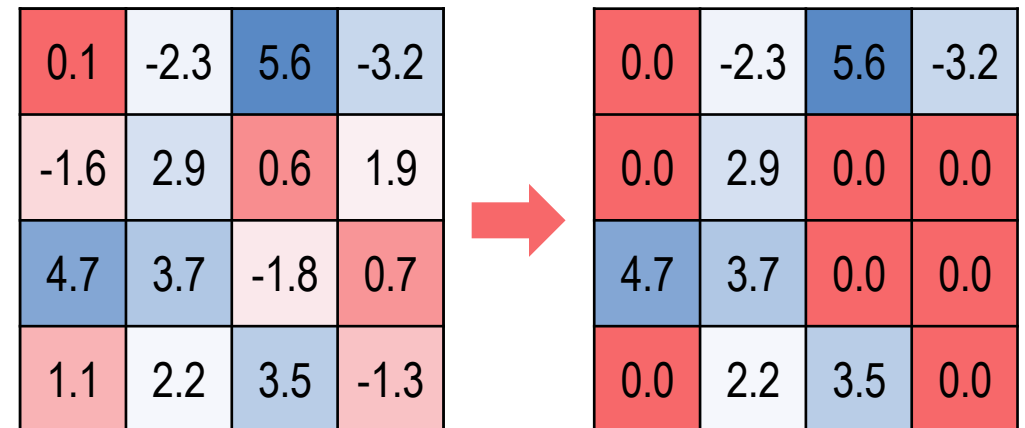


Saliency Score

- The relative magnitudes of the error after pruning
- i.e., the importance score of a weight (or a group of weights)
- Some paper directly use the magnitude of each weight value

➤ Magnitude-based Pruning

Remove weights with absolute value smaller than a threshold
e.g., $|w| < 2.0$



0.1	-2.3	5.6	-3.2
-1.6	2.9	0.6	1.9
4.7	3.7	-1.8	0.7
1.1	2.2	3.5	-1.3

0.0	-2.3	5.6	-3.2
0.0	2.9	0.0	0.0
4.7	3.7	0.0	0.0
0.0	2.2	3.5	0.0



Saliency Score

- The relative magnitudes of the error after pruning
- i.e., the importance score of a weight (or a group of weights)
- Some paper directly use the magnitude of each weight value
 - Magnitude-based Pruning
- Some paper use Hessian matrix
 - Second-Order-based Pruning
 - With higher Hessian trace → more sensitive to pruning
 - e.g., Optimal Brain Damage (LeCun *et al.*, *NIPS* 1989) and Optimal Brain Surgeon (Hassibi *et al.*, *NIPS* 1992)



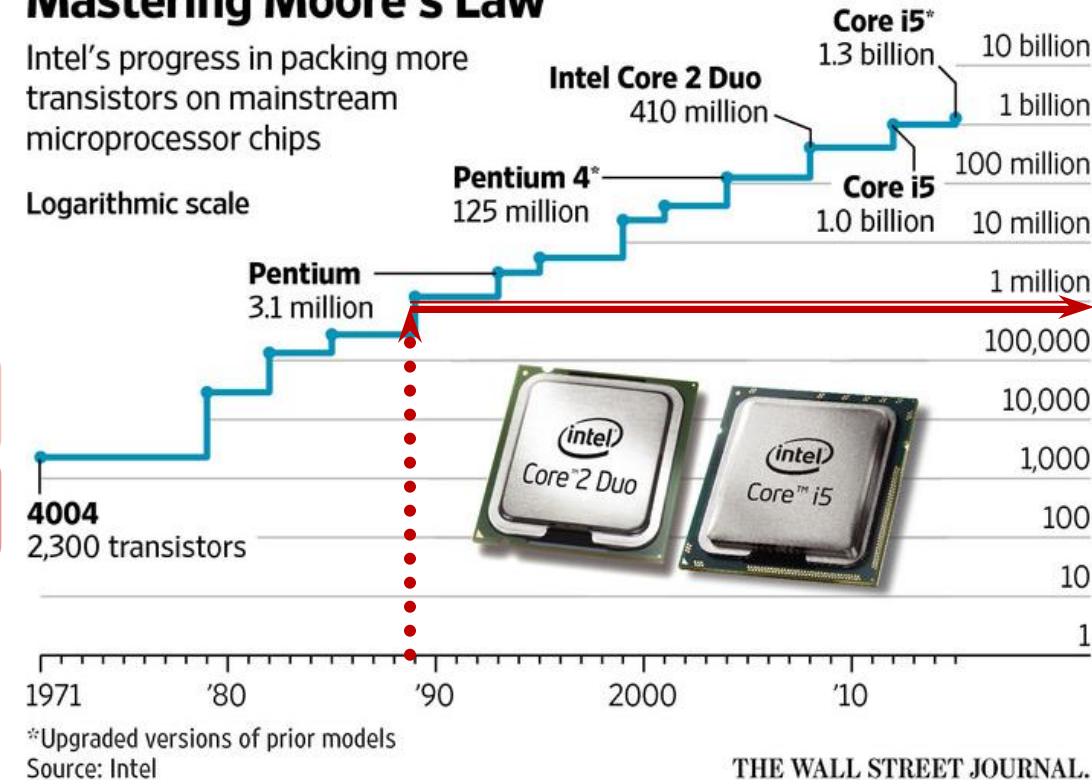
[Recap] History of Artificial Intelligence

- Birth of Artificial Intelligence (1941-1956)
- Early Successes (1956-1974)
- First AI winter (1974-1980)
- Boom (1980-1987)
- Second AI winter (1987-1993)
- AI (1993-2011)
- Deep learning, big data (2011-2020)
- AI Era, artificial general intelligence (2020-present)

Mastering Moore's Law

Intel's progress in packing more transistors on mainstream microprocessor chips

Logarithmic scale



Optimal Brain Damage (LeCun et al., NIPS 1989)

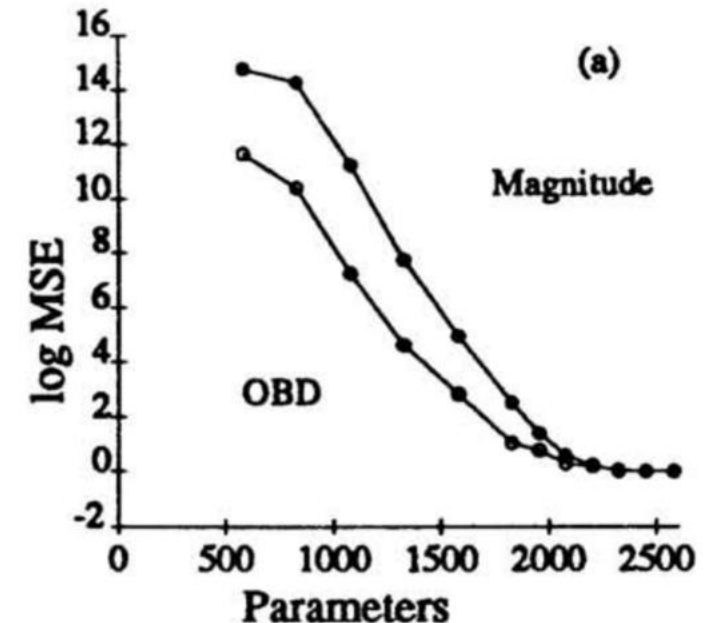
- Saliency score: Hessian (second-order-based pruning)
- Remove redundancy in over-parameterized DNNs and keep the performance/accuracy of the pruned ones

$$\delta E = \sum_i g_i \delta u_i + \frac{1}{2} \sum_i h_{ii} \delta u_i^2 + \frac{1}{2} \sum_{i \neq j} h_{ij} \delta u_i \delta u_j + O(\|\delta \mathbf{u}\|^3)$$

$$g_i = \frac{\partial E}{\partial u_i} \quad \text{and} \quad h_{ij} = \frac{\partial^2 E}{\partial u_i \partial u_j}$$

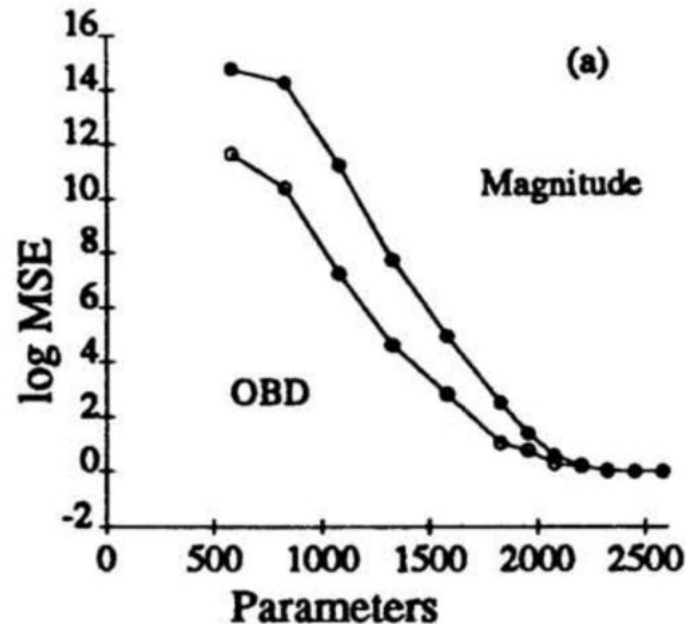
$$\delta E = \left(\frac{\partial E}{\partial \mathbf{w}} \right)^T \cdot \delta \mathbf{w} + \frac{1}{2} \delta \mathbf{w}^T \mathbf{H} \delta \mathbf{w} + O(\|\delta \mathbf{w}\|^3)$$

$\mathbf{H} \equiv \partial^2 E / \partial \mathbf{w}^2$ is the Hessian matrix (containing all second order derivatives)



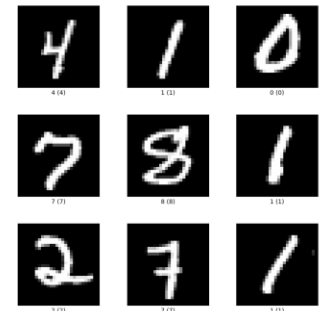
Optimal Brain Damage (LeCun et al., NIPS 1989) (cont.)

- Saliency score: Hessian (second-order-based pruning)
- Remove redundancy in over-parameterized DNNs and keep the performance/accuracy of the pruned ones



Application: handwritten zip code recognition

1011915485726803226414186
6359720299299722510046701
3084111591010615406103631
1064111030475262009979966
8912056708557131427955460
2018750187112991089970984
0109707597331972015519065
1075318255182814358090943
1787521655460354603546055
18255108503047520439401



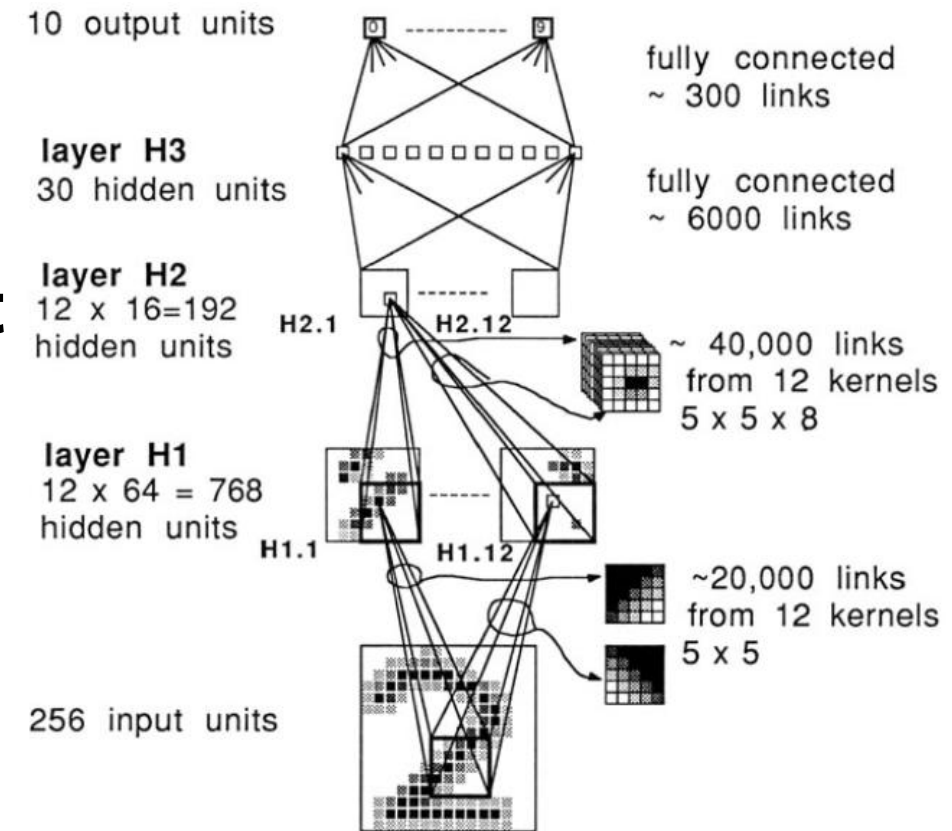
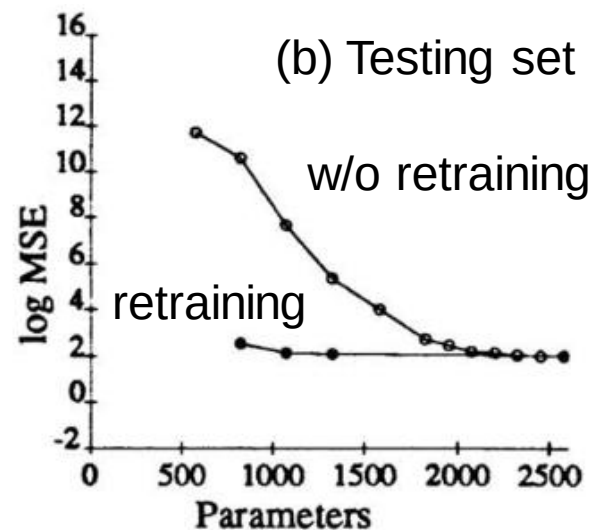
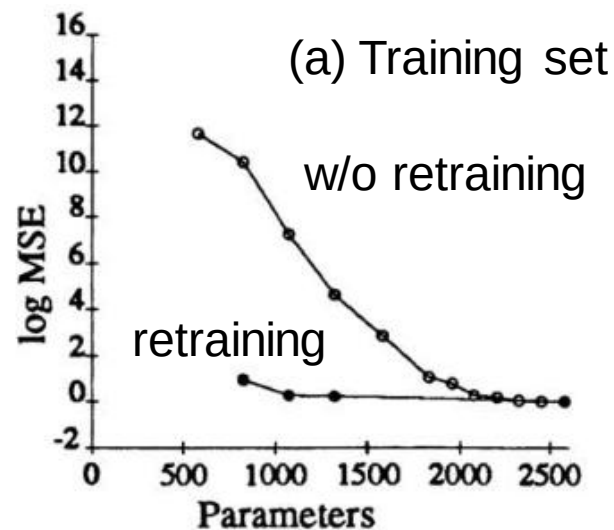
MNIST dataset: 1998

Y. LeCun et al., "Backpropagation applied to handwritten zip code recognition," *Neural computation*, vol. 1, no. 4, 1989.

Y. LeCun et al., "Optimal brain damage," in *Proc. of NIPS*, 1989.

Optimal Brain Damage (LeCun et al., NIPS 1989) (cont.)

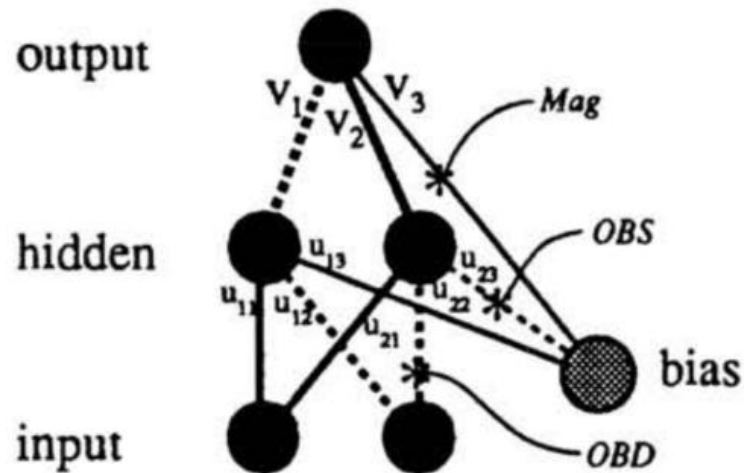
1. Choose a NN architecture
2. Train the NN until get a reasonable solution
3. Compute the saliency score for each weight
4. Delete weights with low saliency score
5. Iterate to 2.



Y. LeCun et al., "Backpropagation applied to handwritten zip code recognition," *Neural computation*, vol. 1, no. 4, 1989.

Optimal Brain Surgeon (Hassibi et al., NIPS 1992)

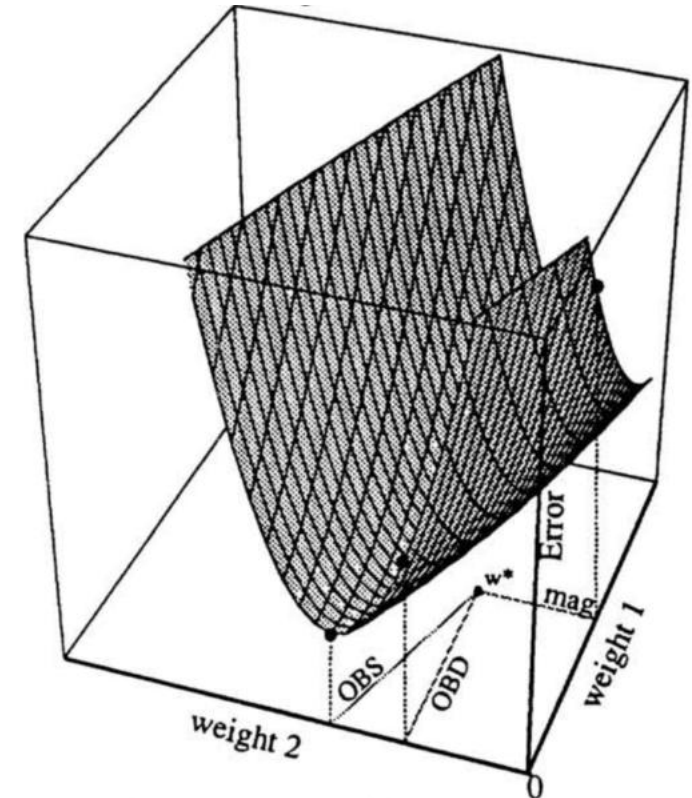
- **Saliency score: inverse Hessian matrix H^{-1}**
- **Motivation:**
 - **A more accurate estimation of saliency score**
 - **Optimal weight updates (minimize the change of objective function here)**
Not minimize the objective function!



Optimal Brain Surgeon (Hassibi et al., NIPS 1992) (cont.)

1. Choose a NN architecture
2. Train the NN until get a reasonable solution
3. Compute the saliency score for each weight
4. Find the weight with smallest saliency score
If the error induced by deleting the weight is much smaller than a threshold E
→ delete the weight and go to 5.
otherwise → go to 6.
5. Update all weights based on calculated δw , and go to 3.
6. Retrain the NN

$$L_q = \frac{1}{2} \frac{w_q^2}{[H^{-1}]_{qq}}$$



Learning Both Weights and Connections for Efficient Neural Networks (Han et al., NIPS 2015)

■ Magnitude-based pruning

- Weights with larger absolute values are more important than other weights
- *Importance* = $|w|$



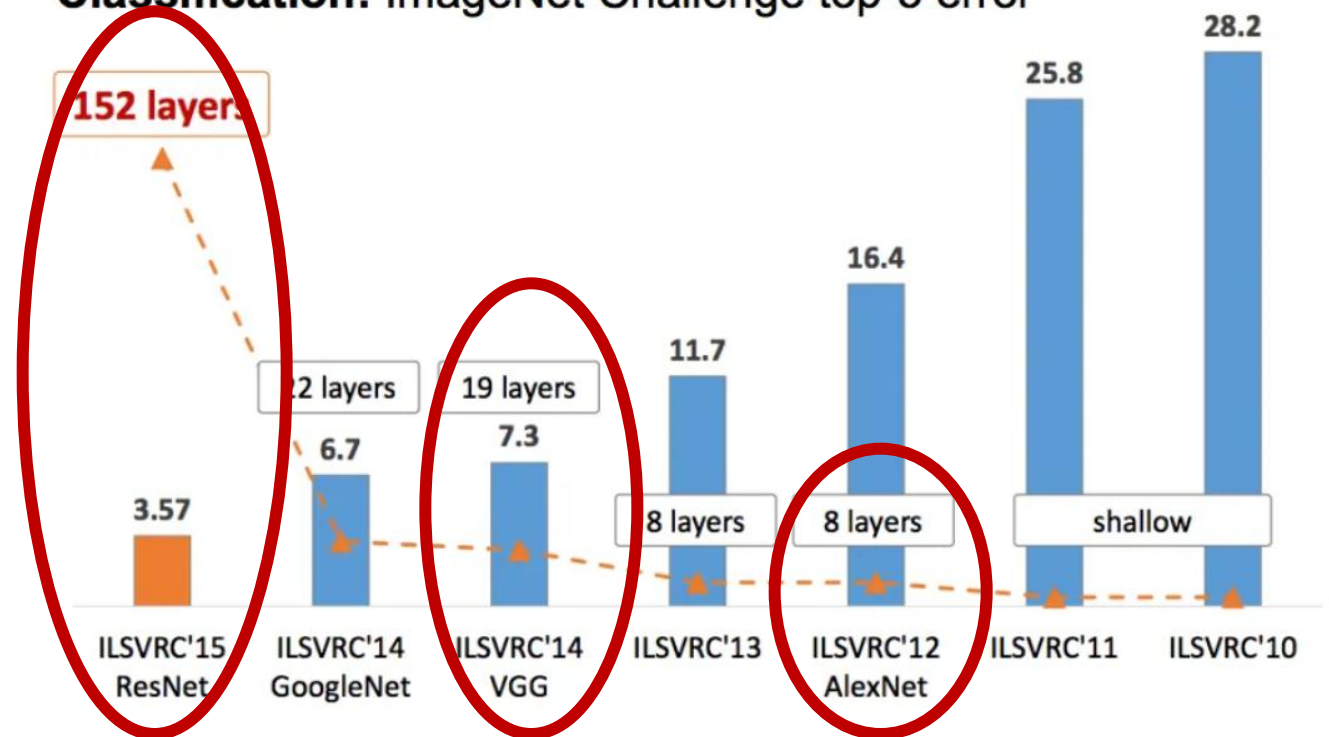
- Remove elements with values below a threshold

0.1	-2.3	5.6	-3.2
-1.6	2.9	0.6	1.9
4.7	3.7	-1.8	0.7
1.1	2.2	3.5	-1.3



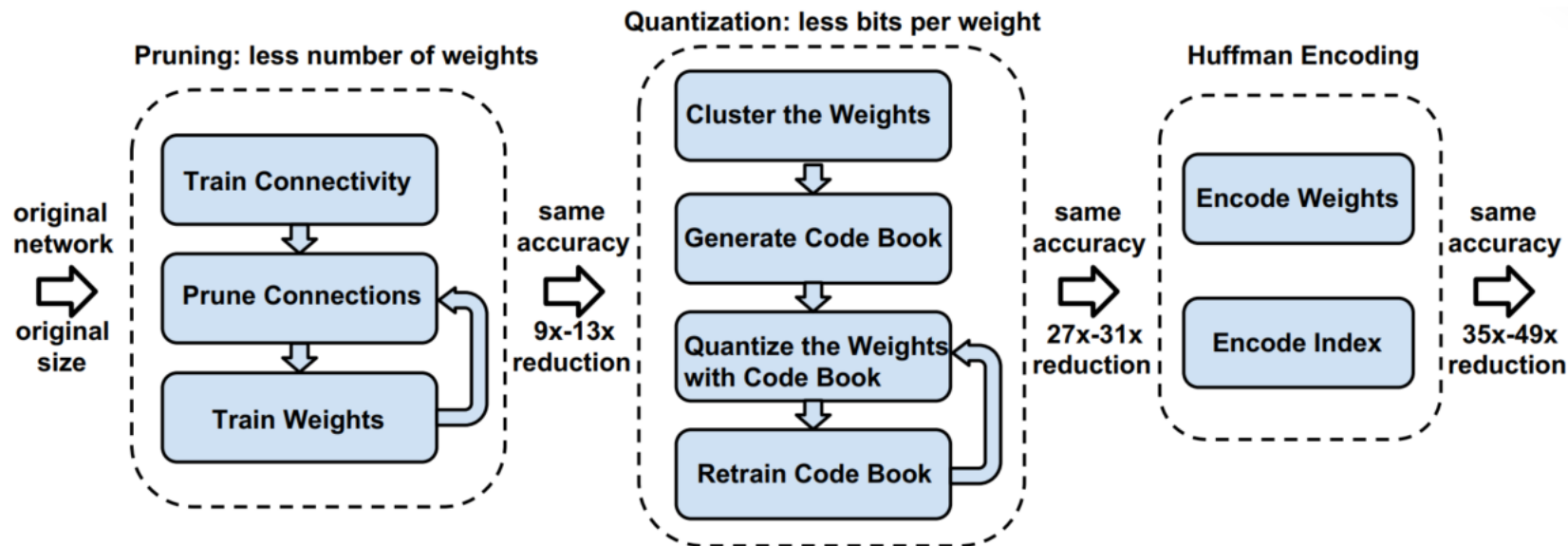
0.0	-2.3	5.6	-3.2
0.0	2.9	0.0	0.0
4.7	3.7	0.0	0.0
0.0	2.2	3.5	0.0

Classification: ImageNet Challenge top-5 error



Deep Compression (Han et al., ICLR 2016)

- **Magnitude-based pruning**
(remove all connections with weights below a threshold)
- + **Quantization (fp32 $\rightarrow$ int8 or lower bit width)**
- + **Huffman encoding**

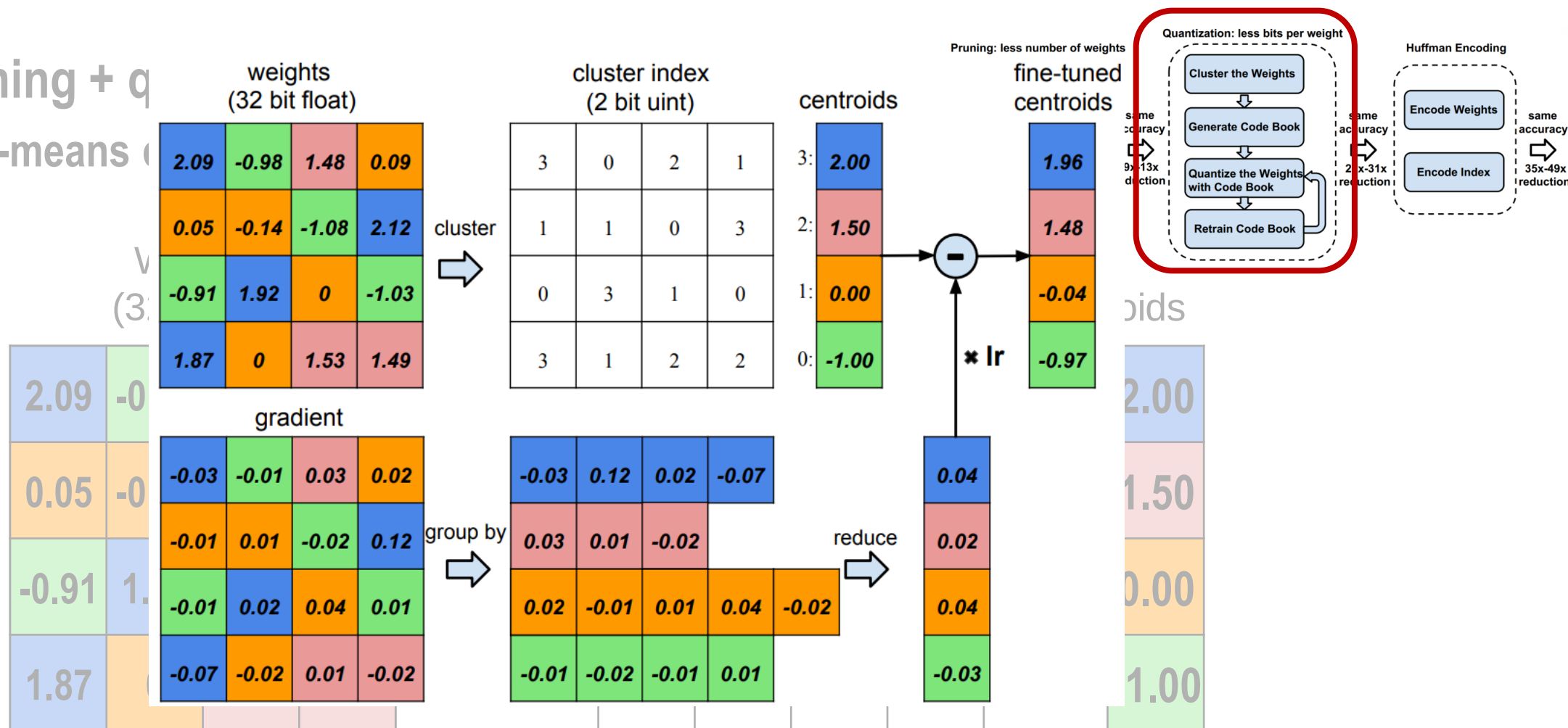


S. Han et al., "deep compression: compressing deep neural networks with pruning, trained quantization and Huffman coding," in *Proc. of ICLR*, 2016.

Deep Compression (Han et al., ICLR 2016) (cont.)

■ Pruning + q

➤ K-means



S. Han et al., "deep compression: compressing deep neural networks with pruning, trained quantization and Huffman coding," in *Proc. of ICLR*, 2016.

Deep Compression (Han et al., ICLR 2016) (cont.)

Table 5: Compression statistics for VGG-16. P: pruning, Q:quantization, H:Huffman coding.

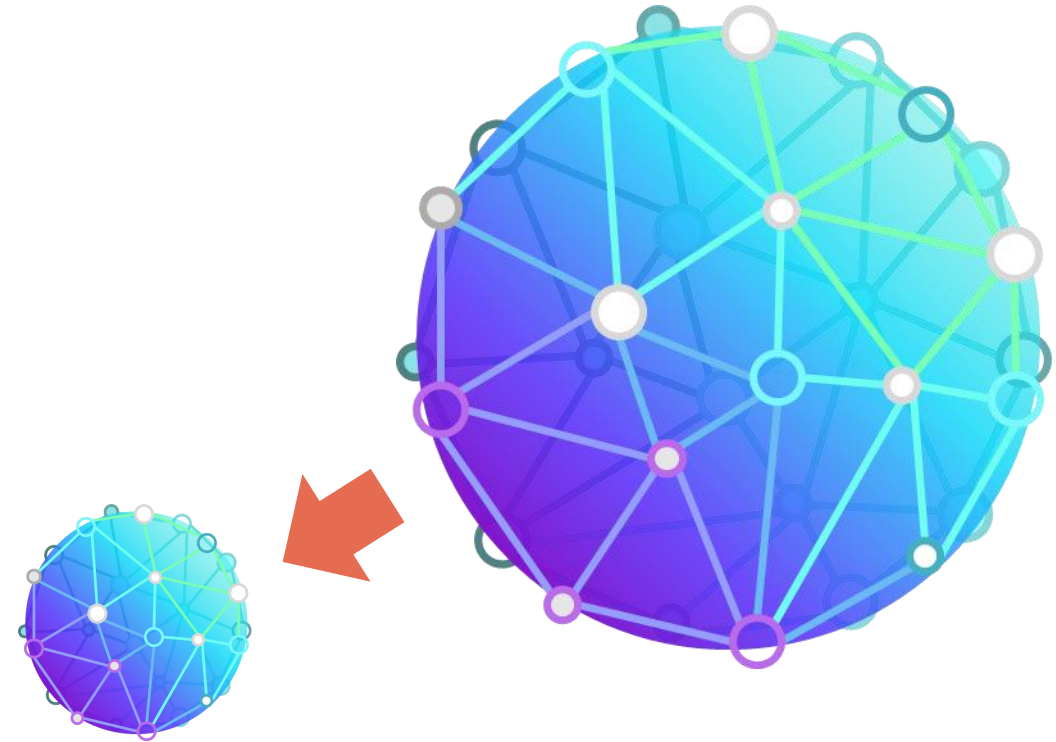
Layer	#Weights	Weights% (P)	Weigh bits (P+Q)	Weight bits (P+Q+H)	Index bits (P+Q)	Index bits (P+Q+H)	Compress rate (P+Q)	Compress rate (P+Q+H)
conv1_1	2K	58%	8	6.8	5	1.7	40.0%	29.97%
conv1_2	37K	22%	8	6.5	5	2.6	9.8%	6.99%
conv2_1	74K	34%	8	5.6	5	2.4	14.3%	8.91%
conv2_2	148K	36%	8	5.9	5	2.3	14.7%	9.31%
conv3_1	295K	53%	8	4.8	5	1.8	21.7%	11.15%
conv3_2	590K	24%	8	4.6	5	2.9	9.7%	5.67%
conv3_3	590K	42%	8	4.6	5	2.2	17.0%	8.96%
conv4_1	1M	32%	8	4.6	5	2.6	13.1%	7.29%
conv4_2	2M	27%	8	4.2	5	2.9	10.9%	5.93%
conv4_3	2M	34%	8	4.4	5	2.5	14.0%	7.47%
conv5_1	2M	35%	8	4.7	5	2.5	14.3%	8.00%
conv5_2	2M	29%	8	4.6	5	2.7	11.7%	6.52%
conv5_3	2M	36%	8	4.6	5	2.3	14.8%	7.79%
fc6	103M	4%	5	3.6	5	3.5	1.6%	1.10%
fc7	17M	4%	5	4	5	4.3	1.5%	1.25%
fc8	4M	23%	5	4	5	3.4	7.1%	5.24%
Total	138M	7.5%(13×)	6.4	4.1	5	3.1	3.2% (31×)	2.05% (49×)

S. Han et al., “deep compression: compressing deep neural networks with pruning, trained quantization and Huffman coding,” in *Proc. of ICLR*, 2016.



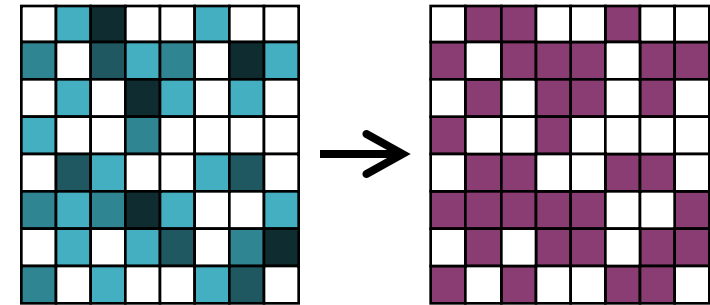
Rethink the Purpose of Pruning

- Pruning is a technique of removing unimportant parameters (weights) of a deep neural network
- Aims to achieve several goals:
 - Reduction in storage (smaller file size)
 - Speed-up during inference (testing)

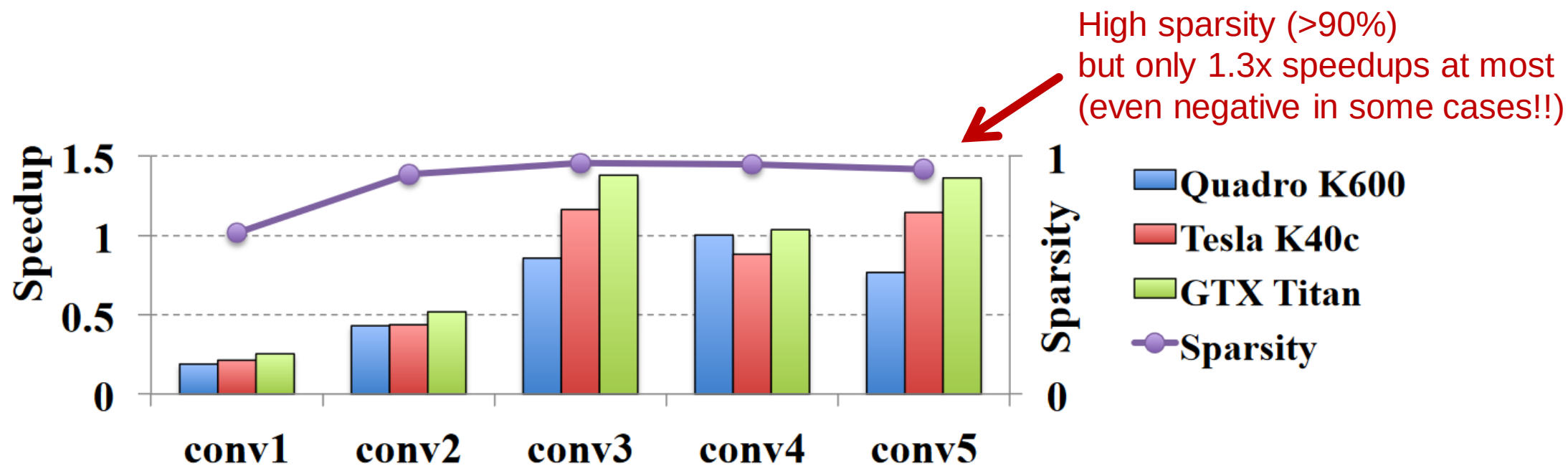


Practical Issues of Weight Pruning

- **Weights become irregular distributed**
 - Thus, also called “unstructured pruning”
- ➔ **Hard to be deployed on devices, hard to speed up**
 - Need complex encoding and decoding
 - Various data length and/or irregular data expression (difficult for data access)
 - Maybe faster on CPU, but slower on GPU
- ➔ **Prune to zero**
 - Actually has the same model size ☹



Relationship Between Sparsity and Speedup



Evaluation speedups of a pruned AlexNet on GPU platforms

- Baseline is profiled by GEMM of cuBLAS.
- The sparse matrixes are stored in the format of Compressed Sparse Row (CSR) and accelerated by cuSPARSE.



Outline

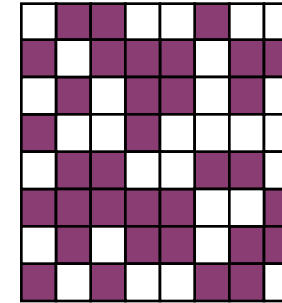
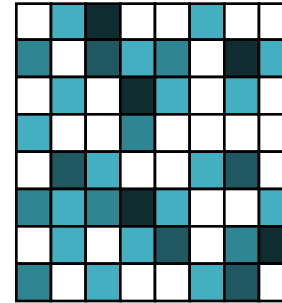
- Motivation of pruning
- Concept of pruning
- Introduction of different pruning methods
 - Unstructured pruning
 - **Structured pruning**
- Hardware for pruned/sparse NNs
- Fine-tuning/training pruned neural networks



Prune the NNs Structurally

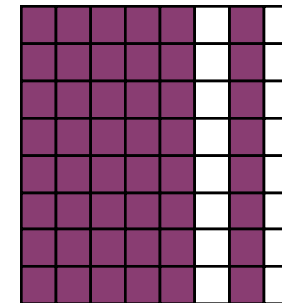
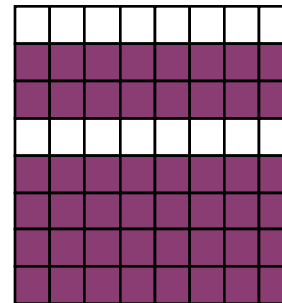
■ Unstructured pruning

- Remove weights with small magnitude



■ Structured pruning

- Remove entire rows/columns/channels



High accuracy



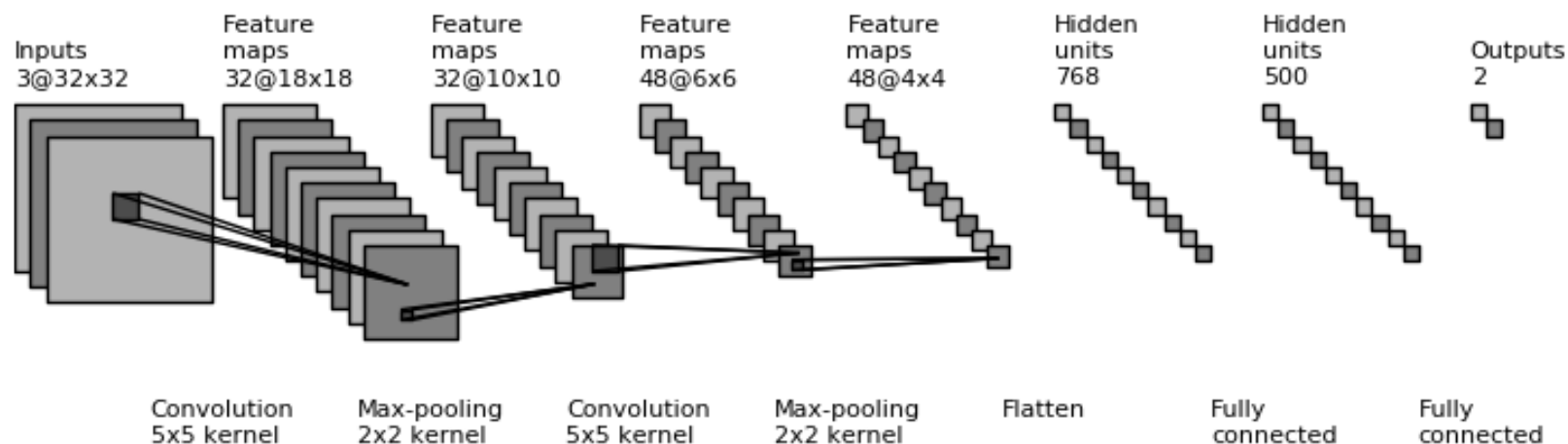
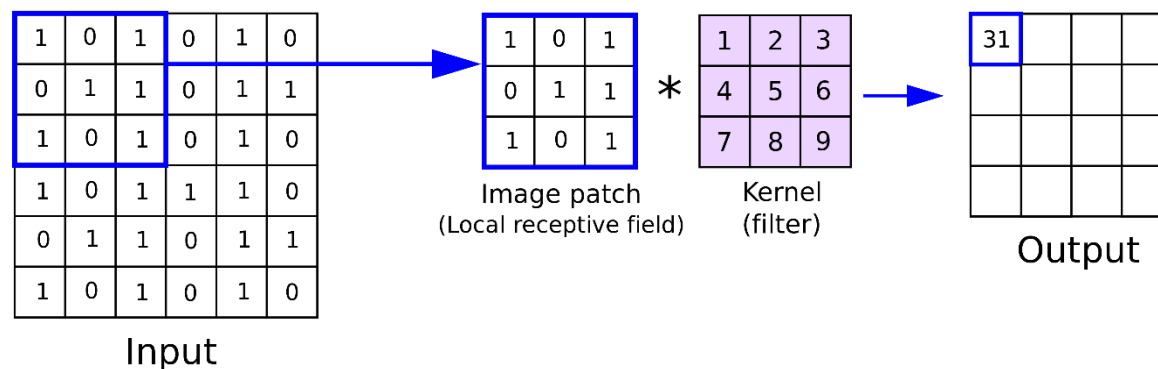
Hardware-friendly
(easier for decoding)



[Review] Dimensions in CNNs

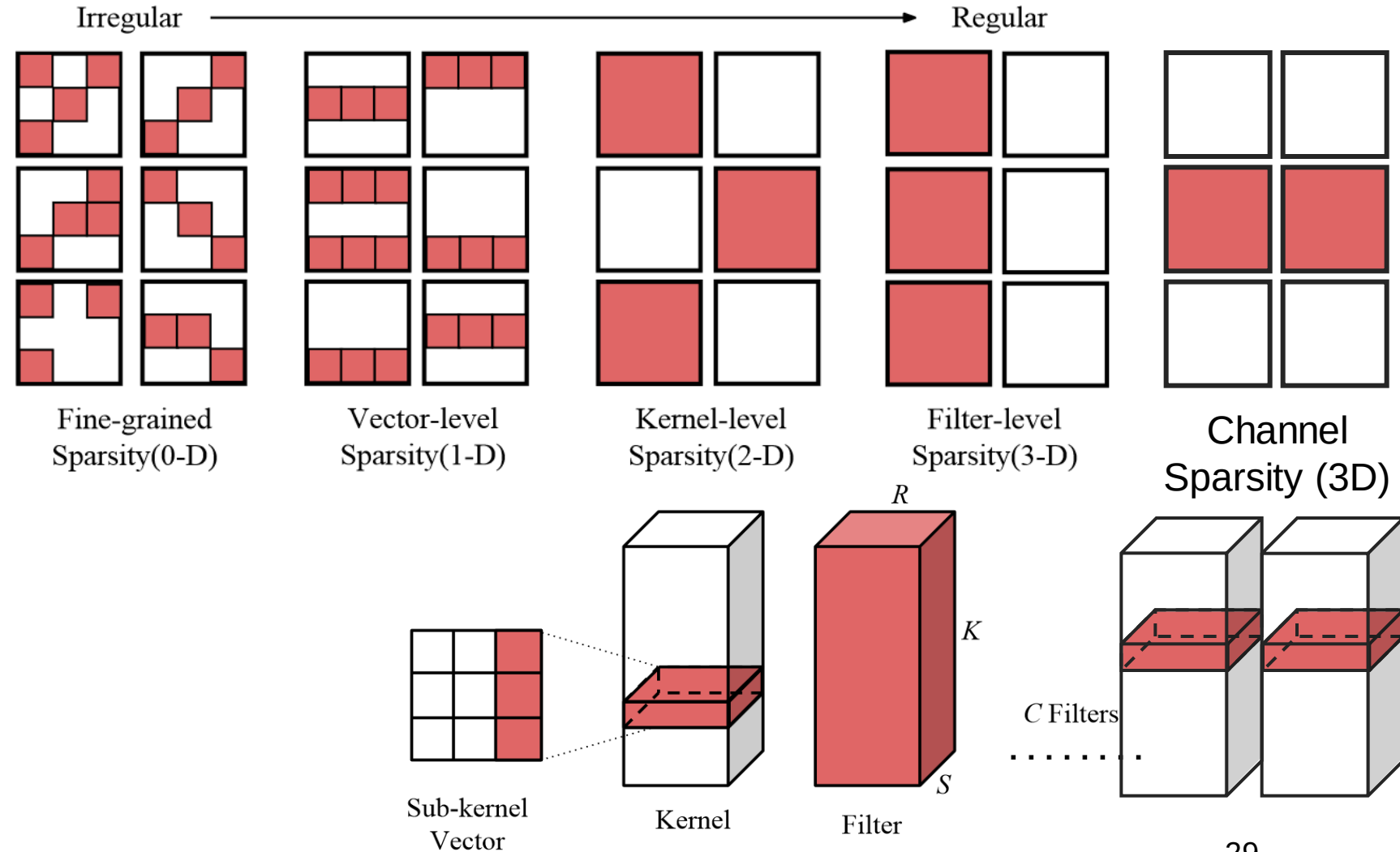
■ The weights of convolutional layers have 4 dimensions $[c_i, c_o, k_h, k_w]$

- c_i : Input channels
- c_o : Output channels
- k_h : Kernel size height
- k_w : Kernel size width



Structured Pruning in CNNs

- Prune kernels
- Prune filters
- Prune channels

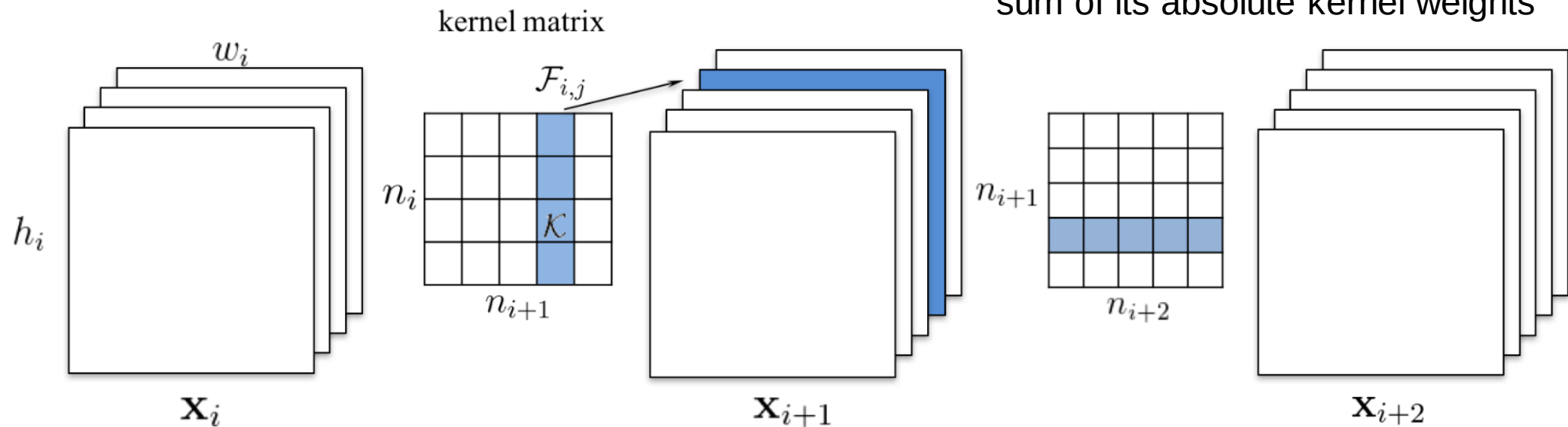


Filter Pruning

- Pruning a filter $F_{i,j}$ results in removal of its corresponding feature map $x_{i+1,j}$ and related kernels in the next layer

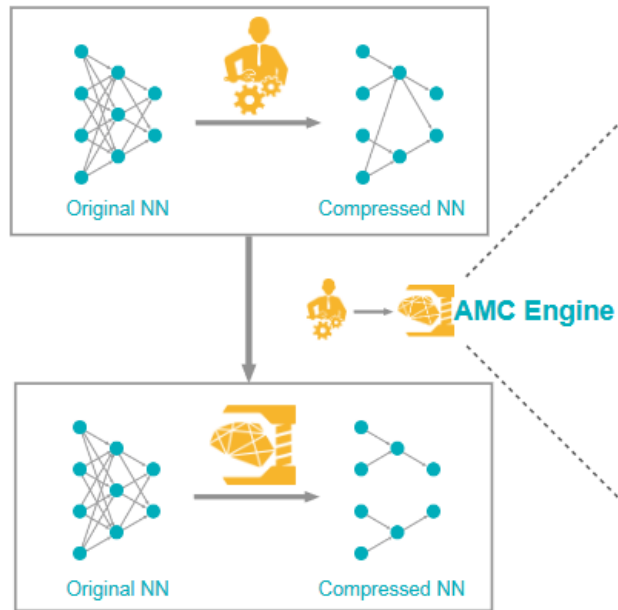
$$s_j = \sum_{l=1}^{n_i} \sum |\mathcal{K}_l|$$

sum of its absolute kernel weights

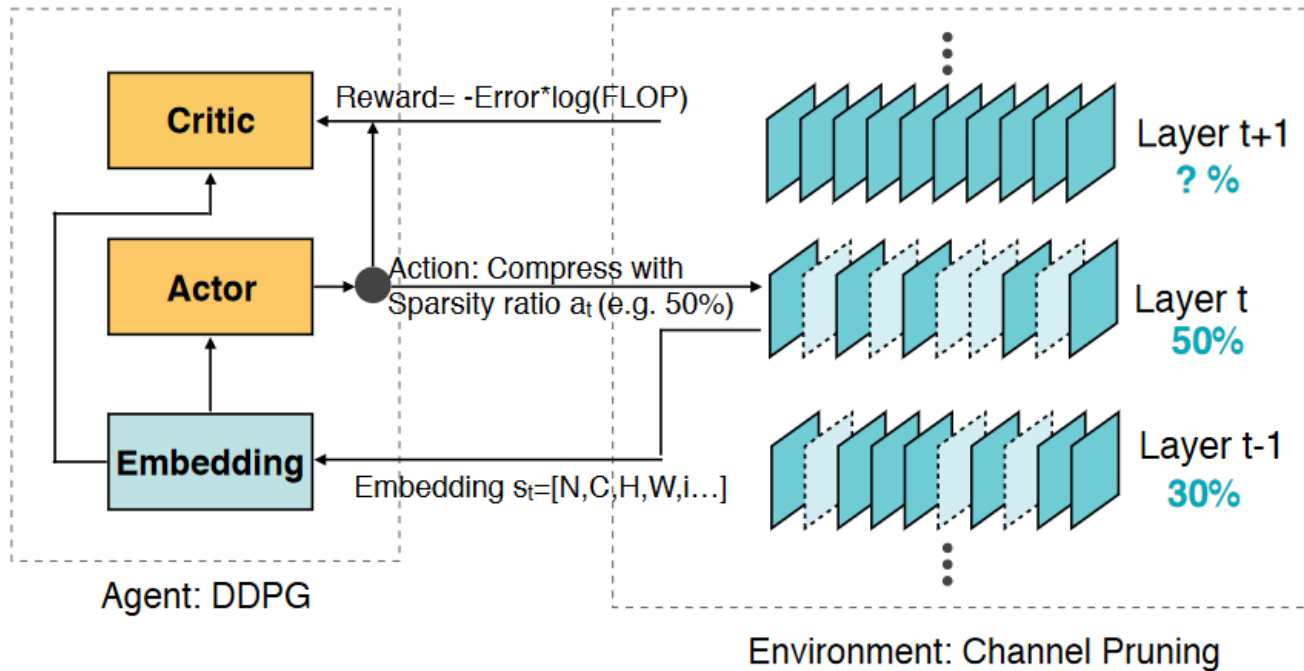


Channel Pruning

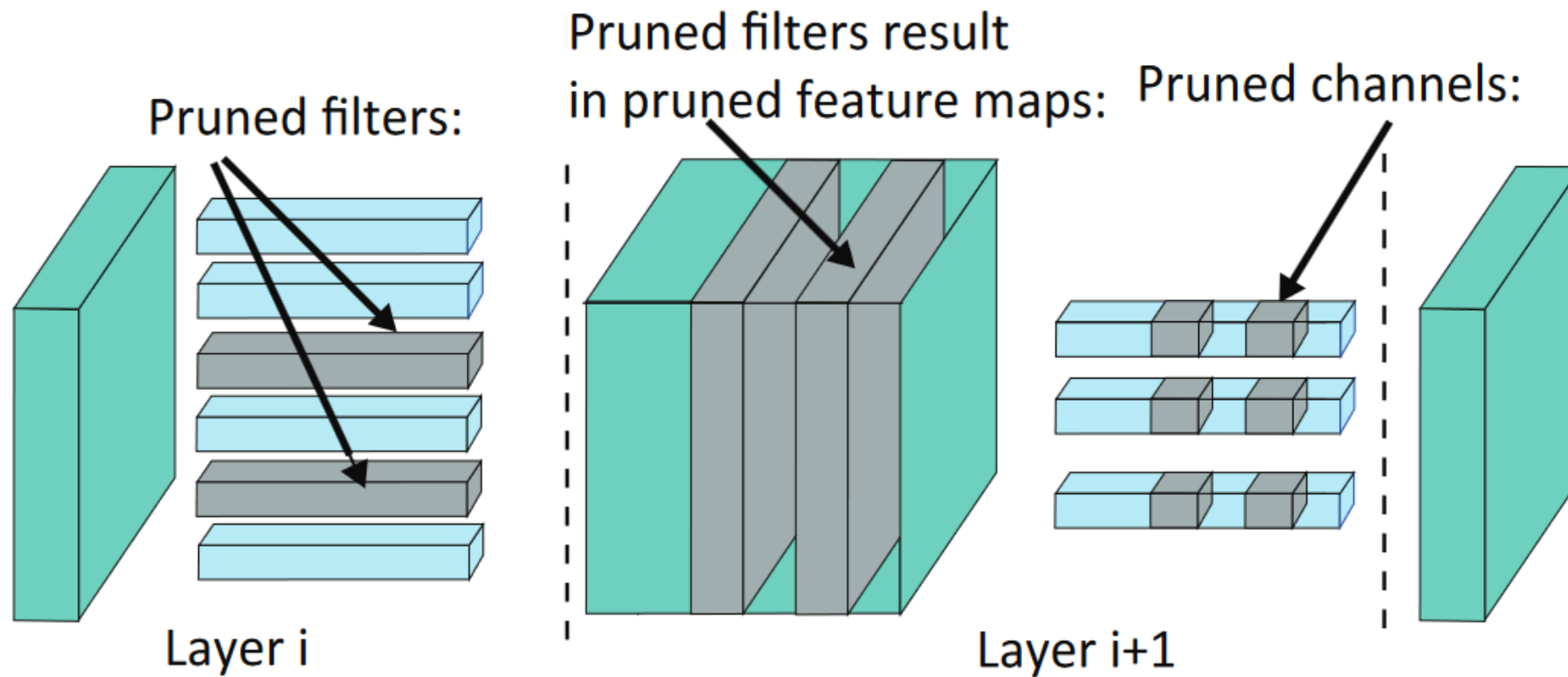
Model Compression by Human:
Labor Consuming, Sub-optimal



Model Compression by AI:
Automated, Higher Compression Rate, Faster



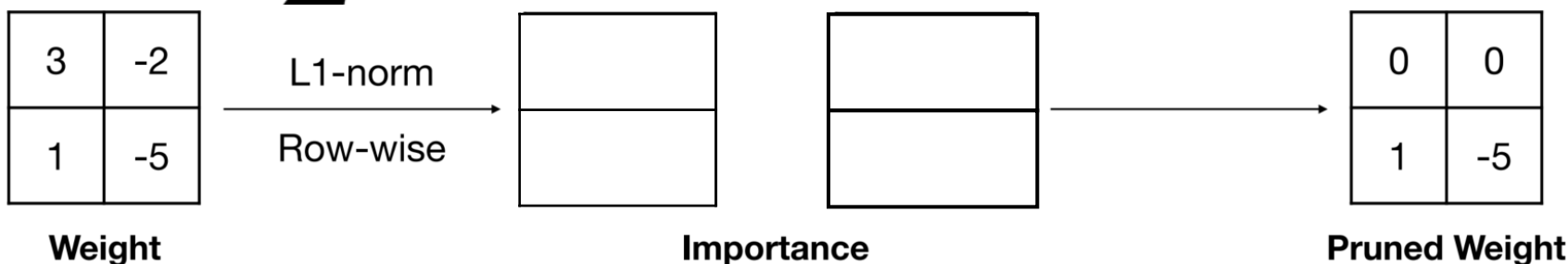
Relationship between Filter Pruning & Channel Pruning



Magnitude-based Structured Pruning

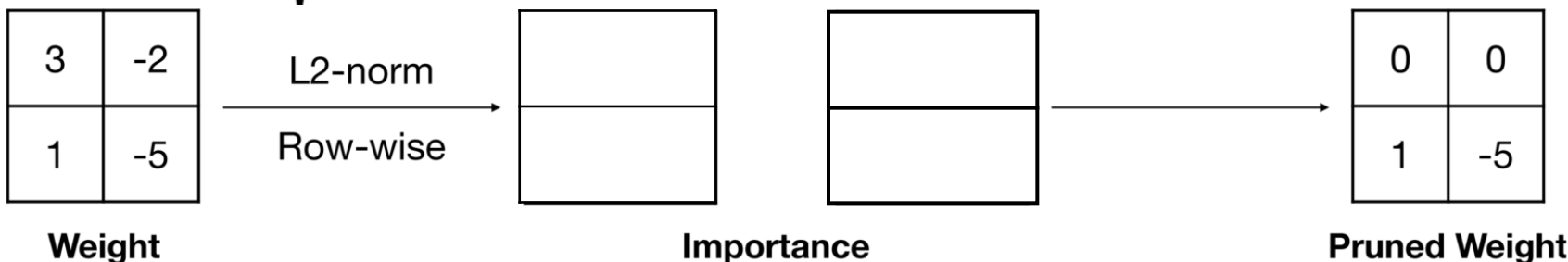
■ L1-norm:

$Importance = \sum |w_i|$, where $\mathbf{W}^{(S)}$ is the structural set S of parameters $\mathbf{W}$



■ L2-norm:

$Importance = \sqrt{\sum_{i \in S} |w_i|^2}$, where $\mathbf{W}^{(S)}$ is the structural set S of parameters $\mathbf{W}$



Percentage-of-Zero-based Pruning

■ Average Percentage of Zeros (APoZ)

M : dimension of output feature map of $O_c^{(i)}$

N : total number of validation examples

$$APoZ_c^{(i)} = APoZ(O_c^{(i)}) = \frac{\sum_k^N \sum_j^M f(O_{c,j}^{(i)}(k) = 0)}{N \times M}$$

Output Activations

Height = 4

Width = 4

0	0.1	0.5	1
1.2	0.6	0.3	0.2
0	0.5	0	0.3
0.2	0	0	0.8

Channel = 3

0.1	0.5	0	0
0.2	0.3	0	1
0.1	0	0	0.5
0.1	0.6	0.7	0.1

Batch = 2

Width = 4

0	0	0.8	0
0.7	0	0.6	0.1
1.2	1	0	0.2
0.5	0	0.3	0.5

Channel = 3

0.1	0.5	0	0
0	0.8	0	1
0.1	0	0.1	1.0
0.2	0	1.0	0

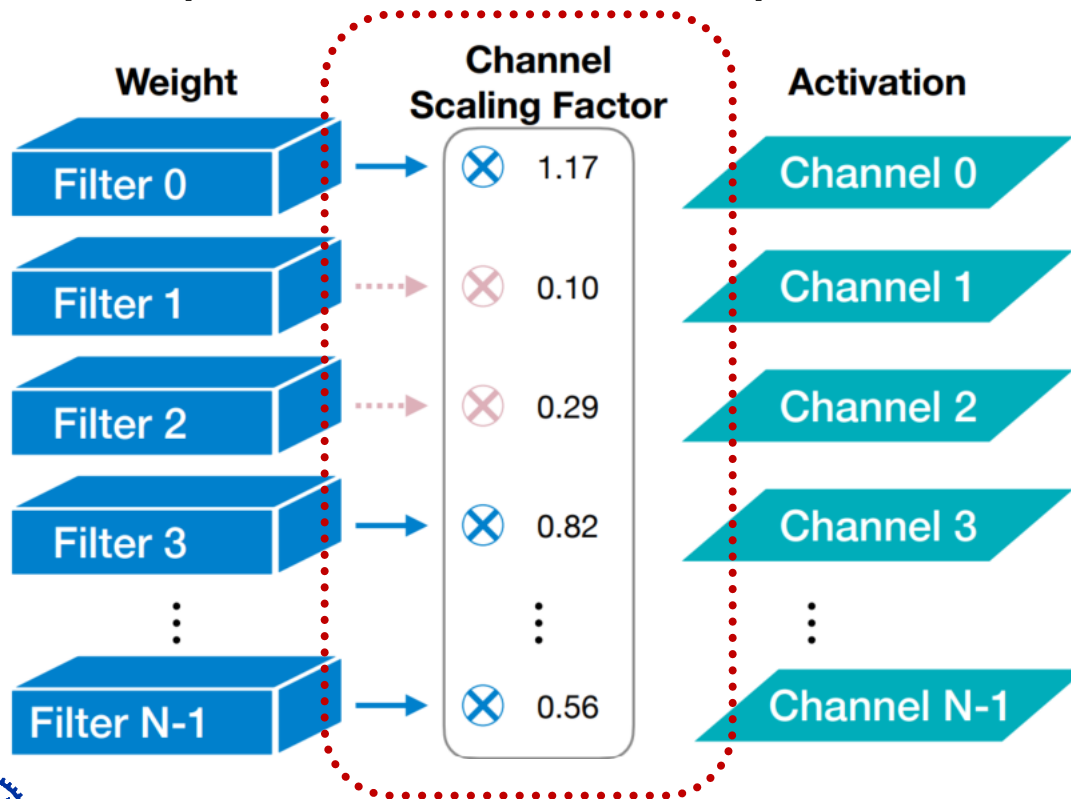
0	0.8	0.1	0
0.2	0	0	0.3
0	0.4	0	0.5
0.2	0	0.3	0

Average Percentage of Zeros (APoZ)



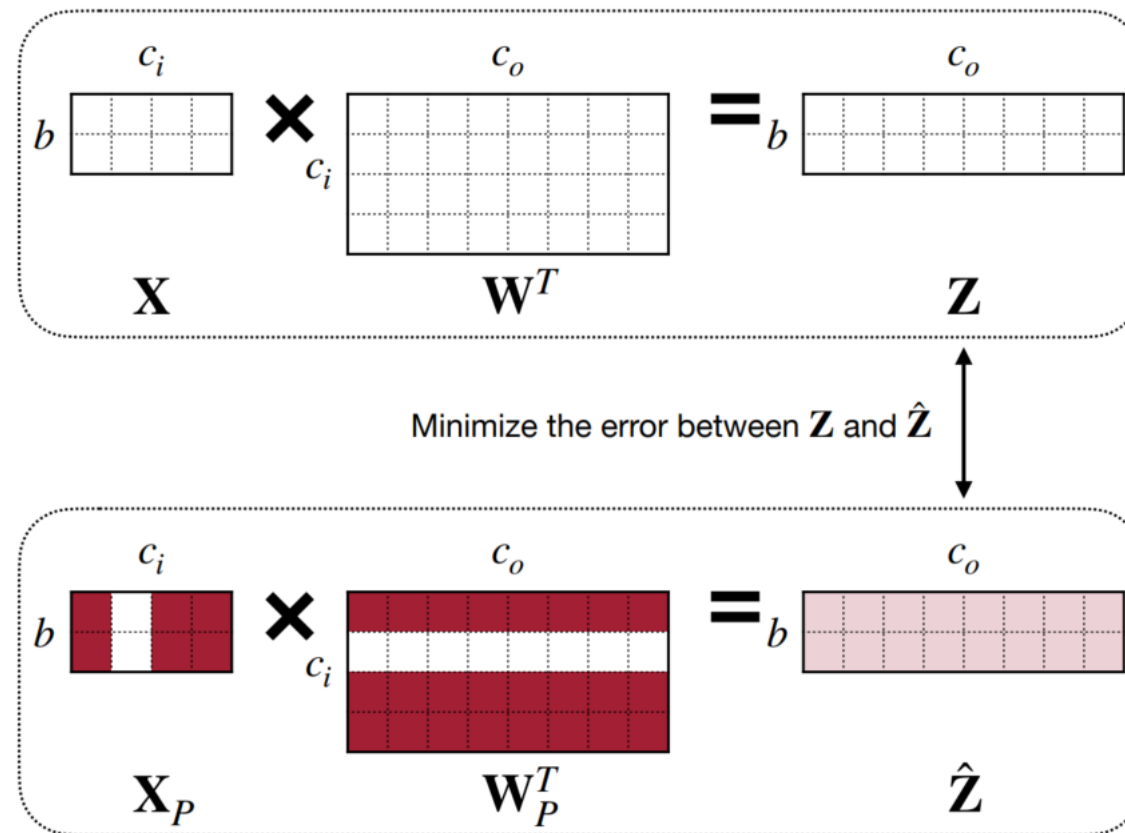
Scaling-based Structured Pruning

- A trainable scaling factor is associated with each filter (i.e., output channel) in convolutional layers



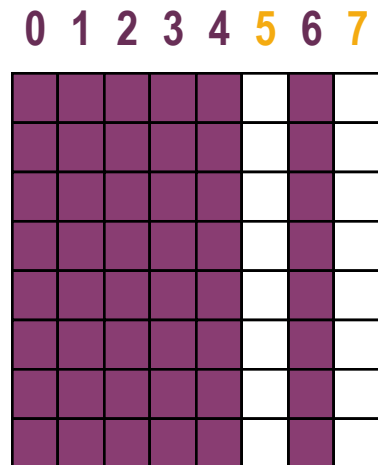
Regression-based Pruning

- Minimize reconstruction error of the corresponding layer's outputs



Pros and Cons of Structured Pruning

- Remove elements in matrices regularly
 - Easy to encode and decode (inference cost ↓)
 - Hard to maintain the performance while using large pruning ratio (accuracy ↓)



Outline

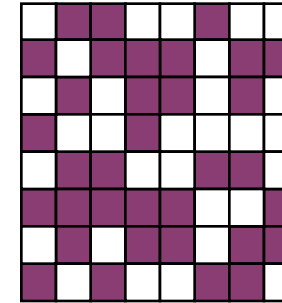
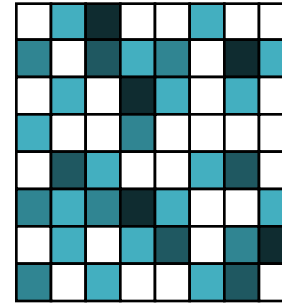
- Motivation of pruning
- Concept of pruning
- Introduction of different pruning methods
 - Unstructured pruning
 - Structured pruning
 - **Semi-structured pruning**
- Hardware for pruned/sparse NNs
- Fine-tuning/training pruned neural networks



Semi-structured Pruning

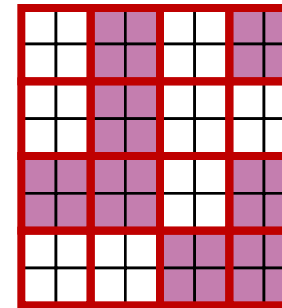
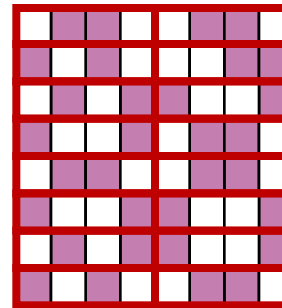
■ Unstructured pruning

- Remove weights with small magnitude



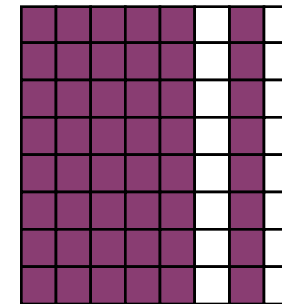
■ Semi-structured pruning

- Depart matrix into small blocks
- Remove some blocks
- Or remove fixed number of weights in each block



■ Structured pruning

- Remove entire rows/columns/channels



High accuracy



Hardware-friendly
(easier for decoding)



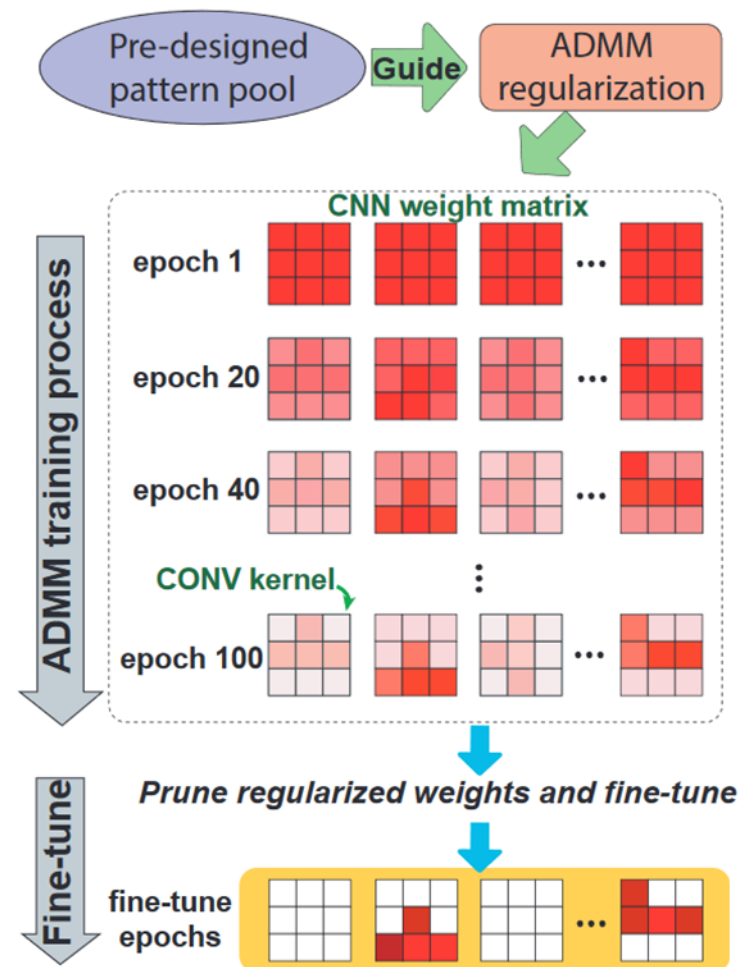
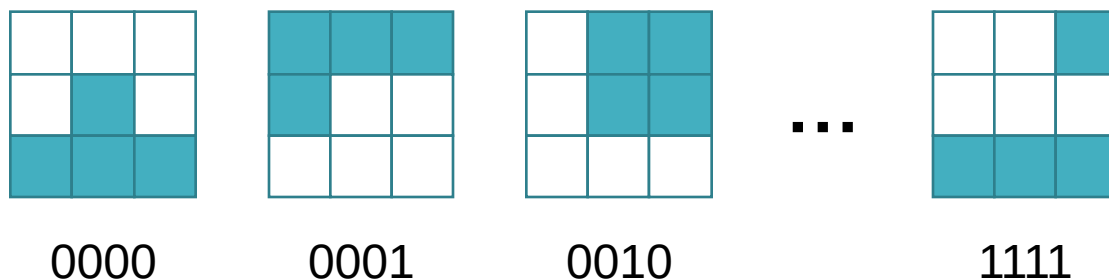
Semi-structured Pruning (cont.)

- **Goal: to find better trade-offs between unstructured pruning and structured pruning**
- **Also called “fine-grained structured pruning”**
- **Including**
 - **Pattern-based pruning**
 - **Block-wise pruning**
 - **(Bank-)Balanced pruning**



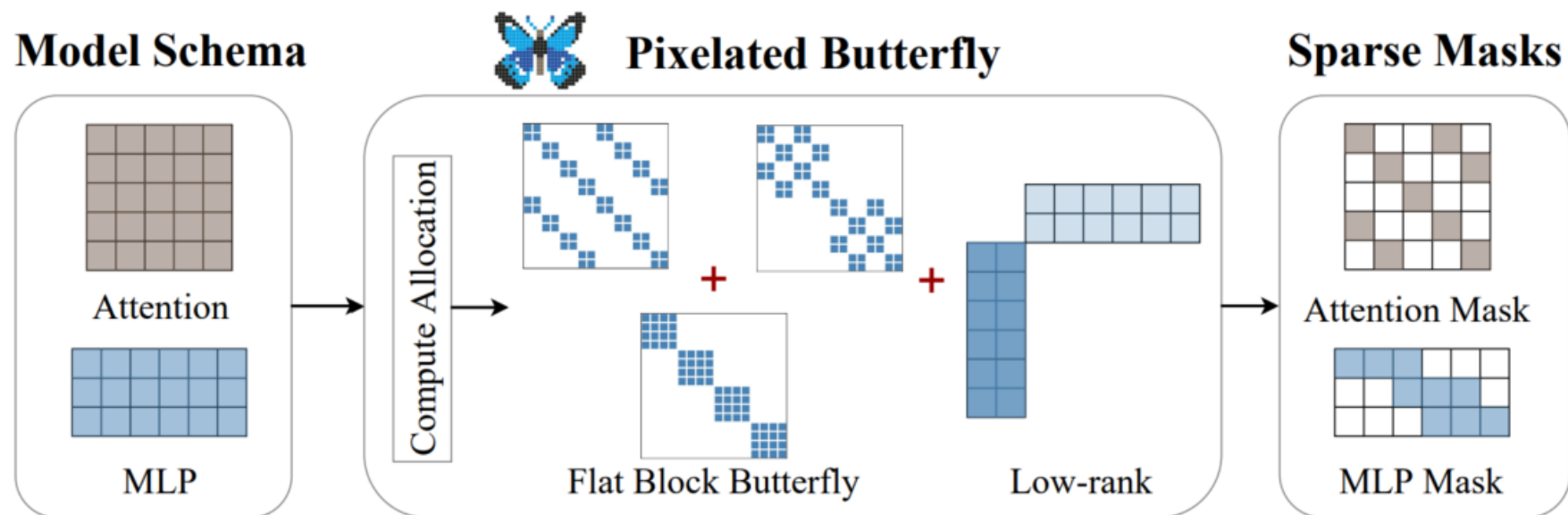
Pattern-based Pruning – 1

- Select those commonly-used patterns for inference
- Encode only the selected weight patterns
 - E.g., if only 16 patterns are commonly-used, the required bits for encoding is 4 ($2^4=16$)



Pattern-based Pruning – 2

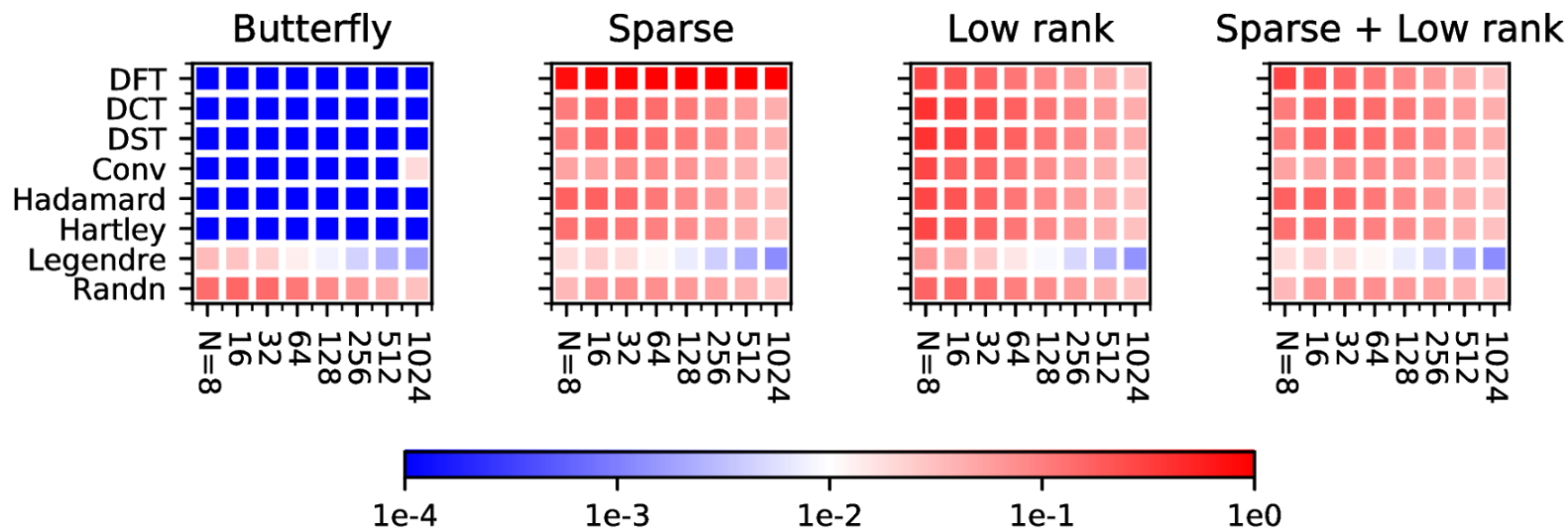
- Decompose matrix into a combination of several patterns
- Pixelfly (Dao *et al.*, ICLR 2022)
 - Butterfly matrices + low-rank approximation



Before Pixelfly (Dao et al., ICML 2019)

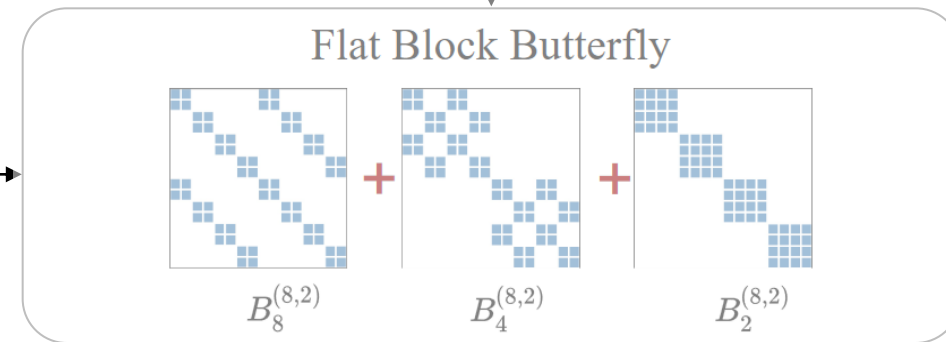
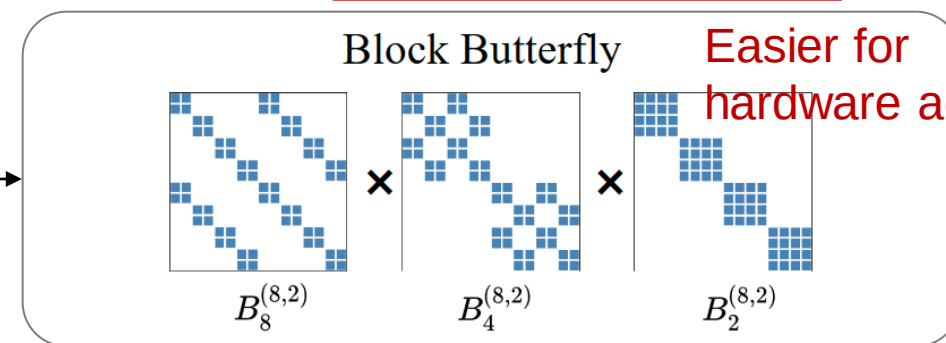
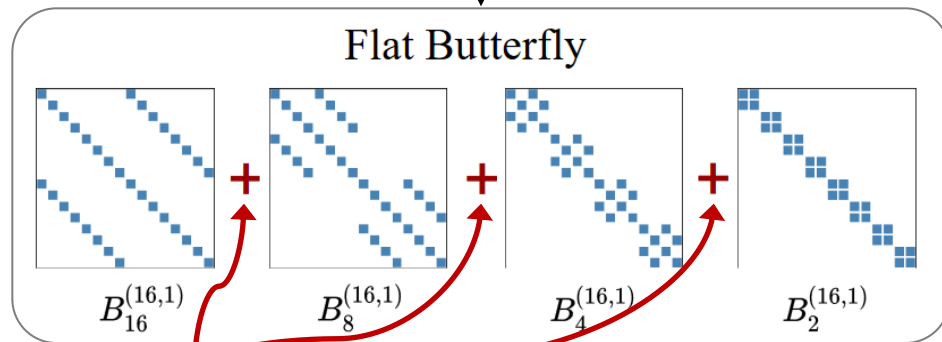
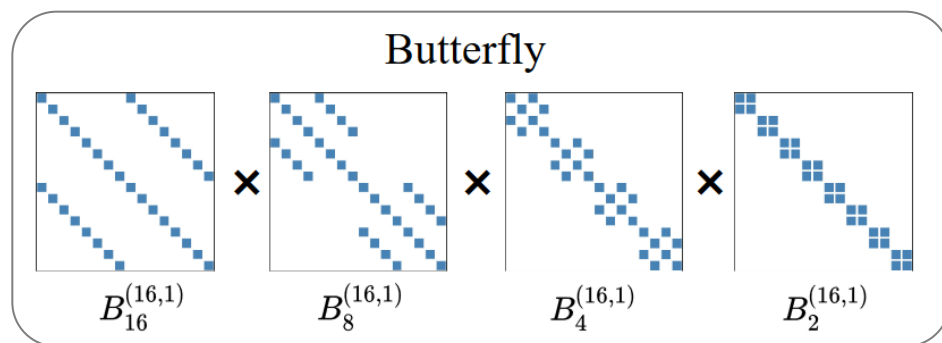
■ Butterfly matrix

- Sparse matrix factorization
- Can express common structured transform
e.g., DFT, DCT, conv, ...
- $O(N \log N)$ multiplication algorithm

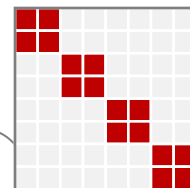


Pixelfly (Dao et al., ICLR 2022)

Flat block butterfly

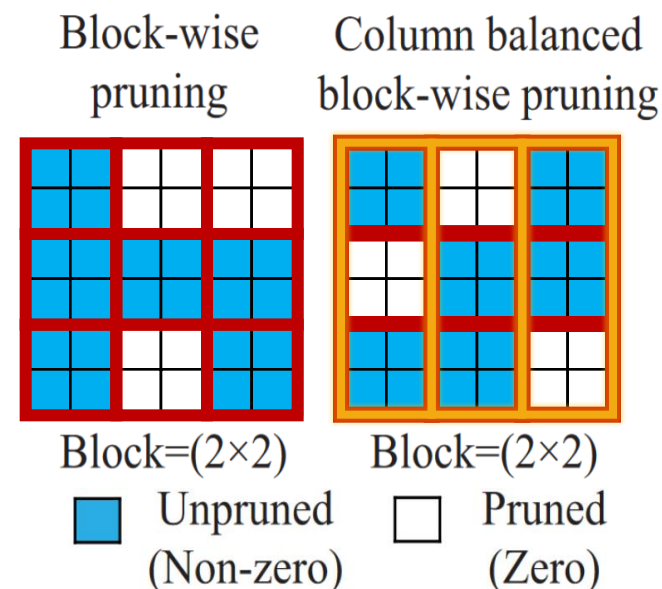
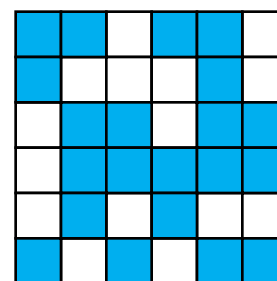


Easier for hardware acceleration!



Block-wise Pruning

- Divide matrix into blocks
 - Remove several blocks
 - Reduce the encoding overhead
 - E.g., 6x6 matrix $\rightarrow$ 36 positions
- Divide into blocks
with each size = $2 \times 2 \rightarrow$ 9 positions



Pruning ratio = 0.33



N:M Sparsity or (Bank-)Balanced Sparsity

- Divide matrix into blocks
- Remove fixed number of elements in each block
 - $N:M \rightarrow$ only N out of M consecutive elements are non-zeros (the other $M - N$ elements are pruned to zeros)

0.2	0.1	0.2	-0.6	0.1	0.4	-0.1	0.6
0.4	-0.3	0.4	0.1	0.2	-0.4	0.1	0.5
0.7	-0.1	-0.3	0.1	0.5	-0.1	0.5	0.1
-0.1	0.6	-0.5	0.3	-0.4	-0.2	0.3	0.6

(a) Original Dense matrix

			-0.6		0.4		0.6
0.4		0.4			-0.4		0.5
0.7				0.5		0.5	
	0.6	-0.5	0.3	-0.4		0.3	0.6

(b) Unstructured sparse matrix by global pruning

		0.2	-0.6			-0.1	0.6
		0.4	0.1			0.1	0.5
0.7	-0.1					0.5	0.1
-0.1	0.6					0.3	0.6

(c) Block sparse matrix by pruning 2x2 blocks according to block average.

		0.2	-0.6		0.4		0.6
0.4		0.4			-0.4		0.5
0.7		-0.3		0.5		0.5	
	0.6	-0.5		-0.4			0.6

(d) Bank-balanced sparse matrix by local pruning inside each 1x4 bank



Outline

- Motivation of pruning
- Concept of pruning
- Introduction of different pruning methods
 - Unstructured pruning
 - Structured pruning
 - Semi-structured pruning
- **Hardware for pruned/sparse NNs**
- Fine-tuning/training pruned neural networks



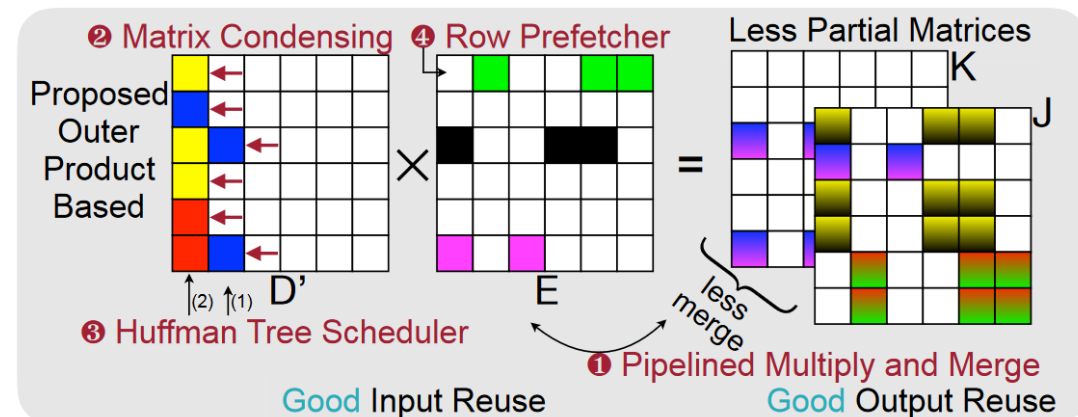
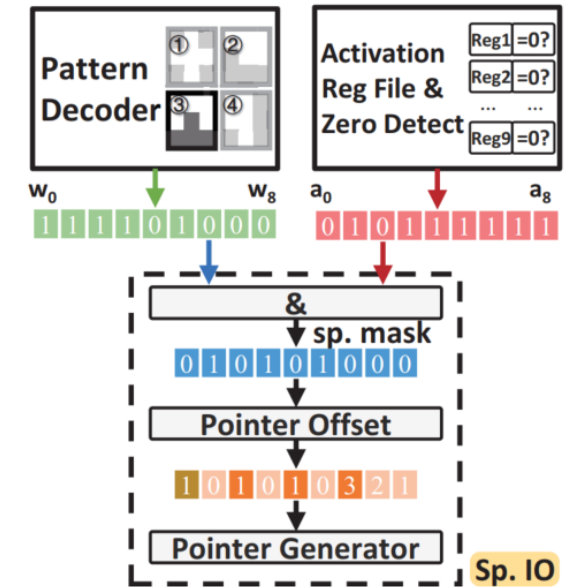
Sparsity Supported by Hardware

■ Support sparse matrix multiplication

- Sparse matrix-matrix multiplication (SpGEMM)

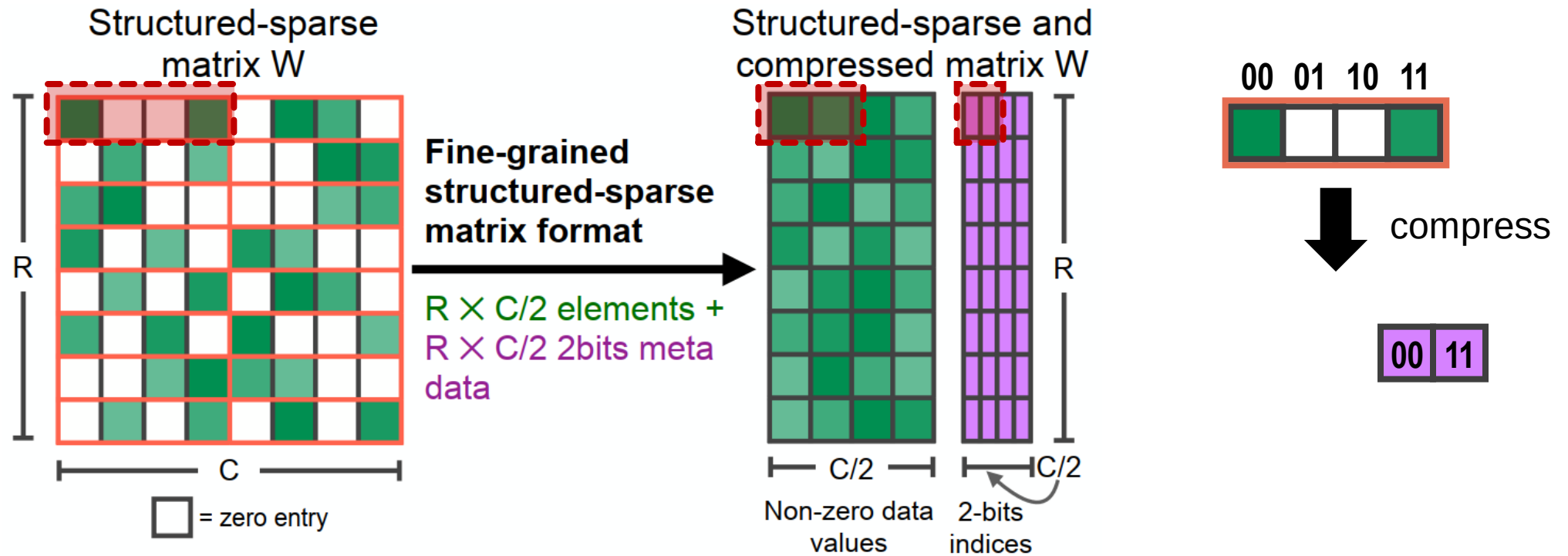
■ Memory access & computing

- Encoding format
- Approach for matrix multiplication



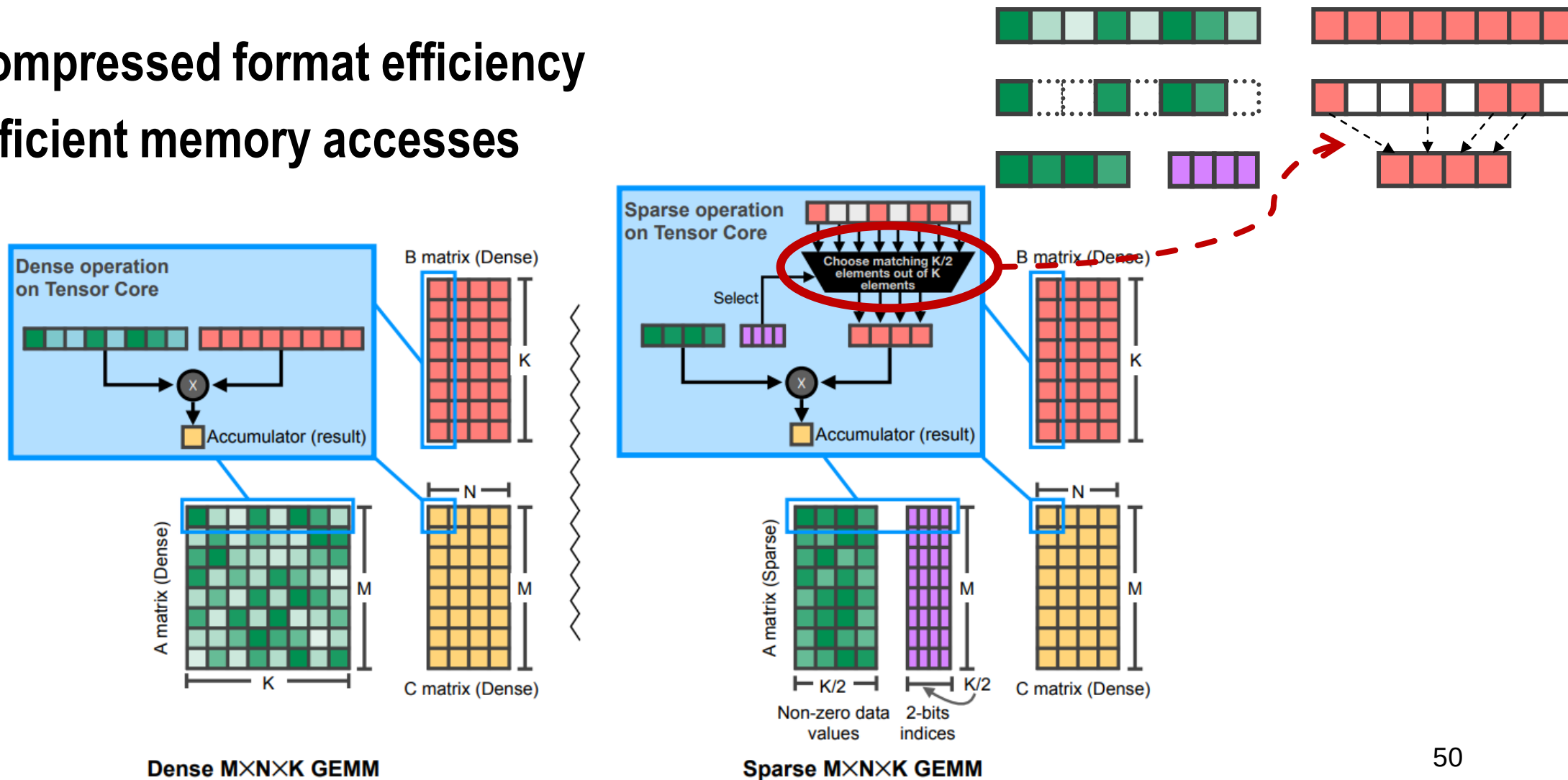
2:4 Sparsity Supported by NVIDIA Ampere GPU

- Compressed format efficiency
- Efficient memory accesses



2:4 Sparsity Supported by NVIDIA Ampere GPU (cont.)

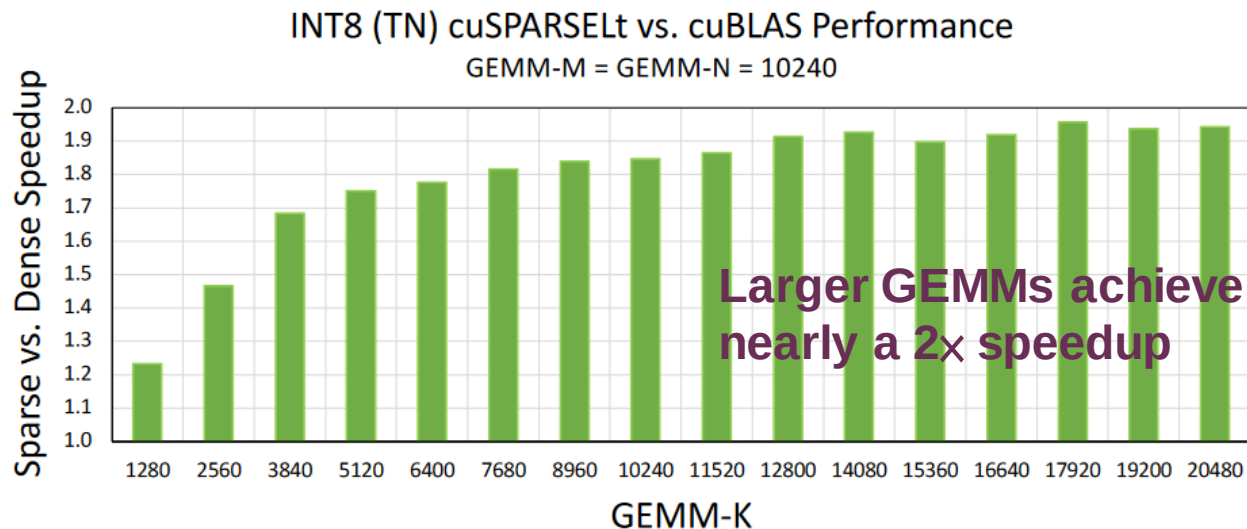
- Compressed format efficiency
- Efficient memory accesses



2:4 Sparsity Supported by NVIDIA Ampere GPU (cont.)

■ 2x math throughput increase

Input Operands	Accumulator	Dense TOPS	Sparse TOPS
FP32	FP32	19.5	-
TF32	FP32	156	312
FP16	FP32	312	624
BF16	FP32	312	624
FP16	FP16	312	624
INT8	INT32	624	1248



Network	Accuracy		
	Dense FP16	Sparse FP16	Sparse INT8
ResNet-34	73.7	73.9	73.7
ResNet-50	76.1	76.2	76.2
ResNet-50 (SWSL)	81.1	80.9	80.9
ResNet-101	77.7	78.0	77.9
ResNeXt-50-32x4	77.6	77.7	77.7
ResNeXt-101-32x16	79.7	79.9	79.9
ResNeXt-101-32x16 (WSL)	84.2	84.0	84.2
DenseNet-121	75.5	75.3	75.3
DenseNet-161	78.8	78.8	78.9
Wide ResNet-50	78.5	78.6	78.5
Wide ResNet-101	78.9	79.2	79.1
Inception v3	77.1	77.1	77.1
Xception	79.2	79.2	79.2
VGG-11	70.9	70.9	70.8
VGG-16	74.0	74.1	74.1
VGG-19	75.0	75.0	75.0
SUNet-128	75.6	76.0	75.4
SUNet-7-128	76.4	76.5	76.3
DRN26	75.2	75.3	75.3
DRN-105	79.4	79.5	79.4

Only conv, fc, and recurrent layers are pruned

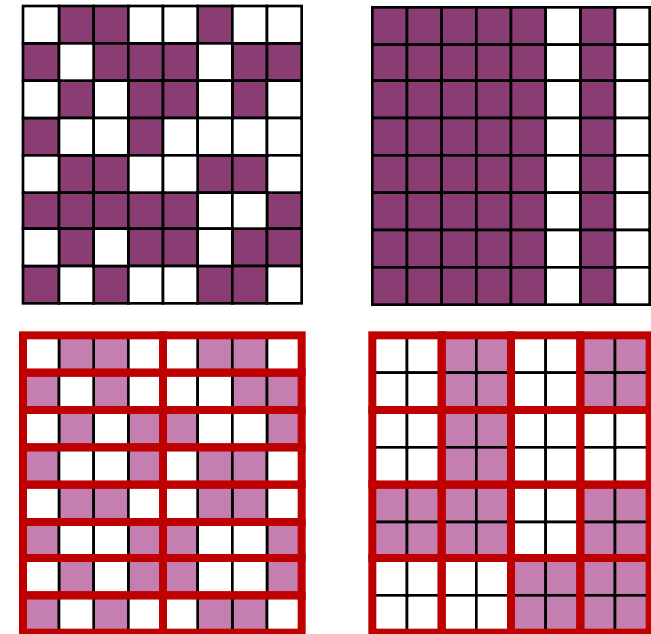


Encoding A Sparse Matrix

- To record the non-zero elements in a sparse matrix

- Encoding formats

- Coordinate format (COO)
- Compressed sparse column (CSC)
- Compressed sparse row (CSR)
- Run-length compression (RLC)
- ...



Encoding Formats

2	0	3
5	0	0
0	0	7

(a) 2D Tensor T
[uncompressed]

val 8b	2	3	5	7
coord-y 2b	0	0	1	2
coord-x 2b	0	2	0	2

(b) COO

val 8b	2	3	5	7
pos 3b	0	2	3	8

(c) COO-1D

val 8b	2	3	5	0	7
run 2b	0	1	0	3	0

(d) RLC

val 8b	2	3	5	7					
flag 1b	1	0	1	1	0	0	0	0	1

(e) Bitmap

val 8b	2	3	5	7
ptr 3b	0	2	3	4
idx 2b	0	2	0	2

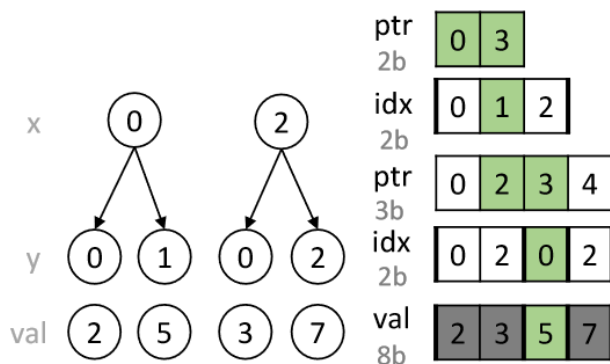
(f) CSR

val 8b	2	5	3	7
ptr 3b	0	2	2	4
idx 2b	0	1	0	2

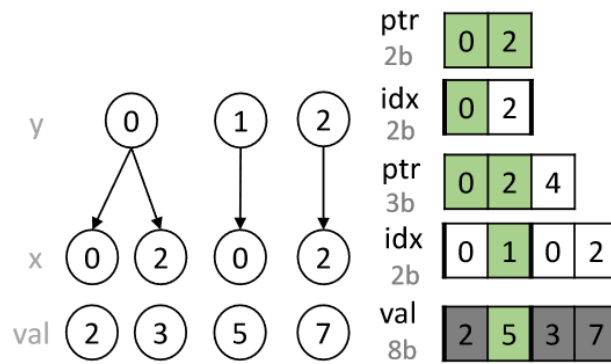
(g) CSC

val 8b	2	5	3	7
ptr 2b	0	2	2	4
idx 1b	0	0	0	1

(h) CSC [relative encoding
of row idx]



(i) CSF (mode-0 tree; y-x compression)



(j) CSF (mode-1 tree; x-y compression)

val 8b	2	5	0	3	0	7
ptr 1b	0	1				
idx 2b	0	2				

(k) Block CSR
[block size: 3x1]

Encoding Formats – Coordinate List (COO)

- Stores absolute positions of non-zeros
- A natural way to express sparse tensors and is used commonly
 - e.g., in PyTorch

	0	1	2
0	2	0	3
1	5	0	0
2	0	0	7

Uncompressed 2D tensor

val 8b	2	3	5	7
coord-y 2b	0	0	1	2
coord-x 2b	0	2	0	2

Encoded 2D tensor

	0	1	2
0	2	0	3
1	5	0	0
2	0	0	7

Decoded 2D tensor



Encoding Formats – Compressed Sparse Row (CSR)

■ Compresses a matrix by processing each row as a sparse vector

- *val* contains all non-zero values (ordered row-wise) 
- *idx* stores their column indices
- *ptr* informs about the total non-zeros above row i (by $ptr[i + 1] - ptr[i]$)

	0	1	2
0	2	0	3
1	5	0	0
2	0	0	7

Uncompressed 2D tensor

val 8b	2	3	5	7
ptr 3b				
idx 2b	0	2	0	2

Encoded 2D tensor

	0	1	2
0			
1			
2			

2 5 3 7

Decoded 2D tensor



Encoding Formats – Compressed Sparse Row (CSR)

■ Compresses a matrix by processing each row as a sparse vector

- *val* contains all non-zero values (ordered row-wise) 
- *idx* stores their column indices
- *ptr* informs about the total non-zeros above row *i* (by $ptr[i + 1] - ptr[i]$)

	0	1	2
0	2	0	3
1	5	0	0
2	0	0	7

Uncompressed 2D tensor

val 8b	2	3	5	7
ptr 3b	0	2	3	4
idx 2b	0	2	0	2

Encoded 2D tensor

	0	1	2
0	2	0	3
1	5	0	0
2	0	0	7

Decoded 2D tensor

Encoding Formats – Compressed Sparse Column (CSC)

- similar to CSR, except that non-zeros are stored column-wise
 - *val* contains non-zeros (organized column-wise) 
 - *idx* stores their row indices
 - *ptr* informs about the total non-zeros in above column *i* (by $ptr[i + 1] - ptr[i]$)

	0	1	2
0	2	0	3
1	5	0	0
2	0	0	7

Uncompressed 2D tensor

val 8b	2	5	3	7
ptr 3b				
idx 2b	0	1	0	2

Encoded 2D tensor

	0	1	2	
0				2 3
1				5
2				7

Decoded 2D tensor



Encoding Formats – Compressed Sparse Column (CSC)

- similar to CSR, except that non-zeros are stored column-wise
 - *val* contains non-zeros (organized column-wise) 
 - *idx* stores their row indices
 - *ptr* informs about the total non-zeros in above column *i* (by $ptr[i + 1] - ptr[i]$)

	0	1	2
0	2	0	3
1	5	0	0
2	0	0	7

Uncompressed 2D tensor

val 8b	2	5	3	7
ptr 3b	0	2	2	4
idx 2b	0	1	0	2

Encoded 2D tensor

	0	1	2	
0	2	0	3	2 3
1	5	0	0	5
2	0	0	7	7

Decoded 2D tensor



Outline

- Motivation of pruning
- Concept of pruning
- Introduction of different pruning methods
 - Unstructured pruning
 - Structured pruning
 - Semi-structured pruning
- **Hardware for pruned/sparse NNs**
 - **If multiple sparse configurations can be supported...**
- Fine-tuning/training pruned neural networks



Sparsity Acceleration

- Know sparse configurations supported by an accelerator
 - E.g., N:M sparsity with $M = 4$, $N = 1, 2$, or 4
- Know how a accelerator computes SpGEMM
 - E.g., inner product

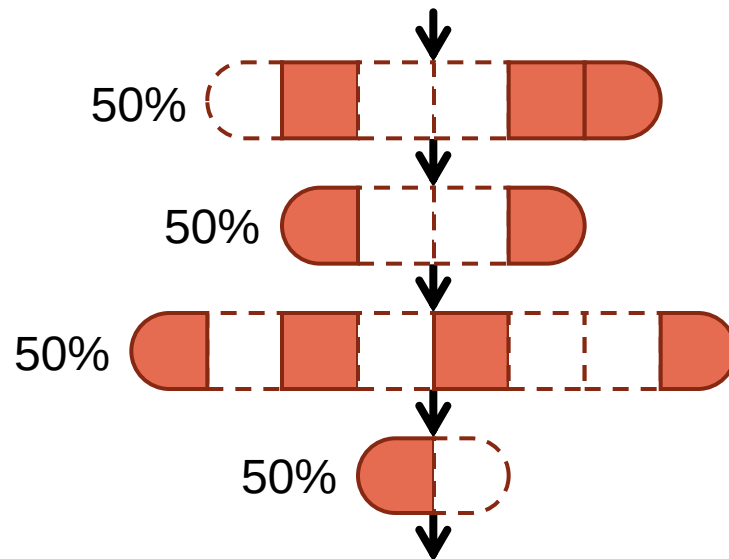


**Next, obtain better trade-off between accuracy and speedup
by finding suitable sparse setting & mask**



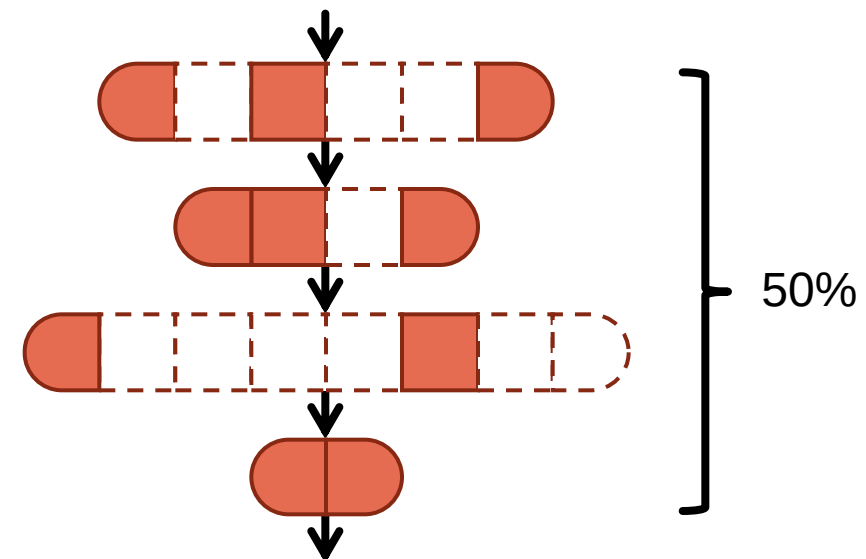
Local Pruning & Global Pruning

- Local pruning applies the same rate to each layer
- Global pruning applies the compression ratio on the whole network



Local pruning

(Uniform pruning)

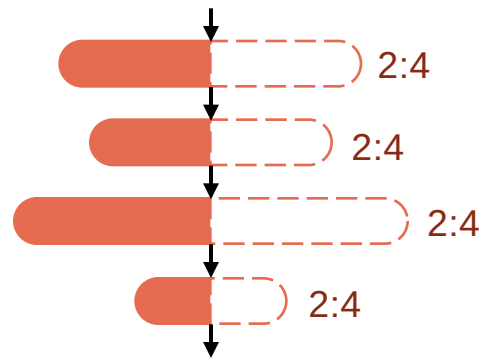


Global pruning



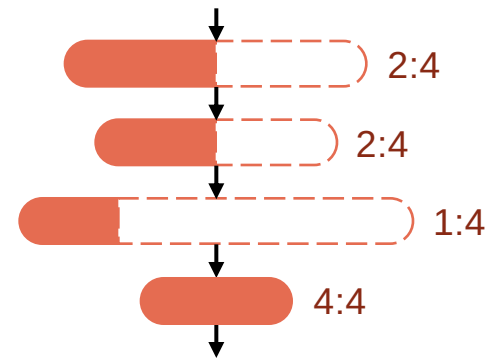
Layer-wise Pruning

- More flexible
 - Better tradeoff between accuracy and sparsity
- Need configurable hardware supported
- Need to find the layer-wise sparse setting



Uniform 2:4

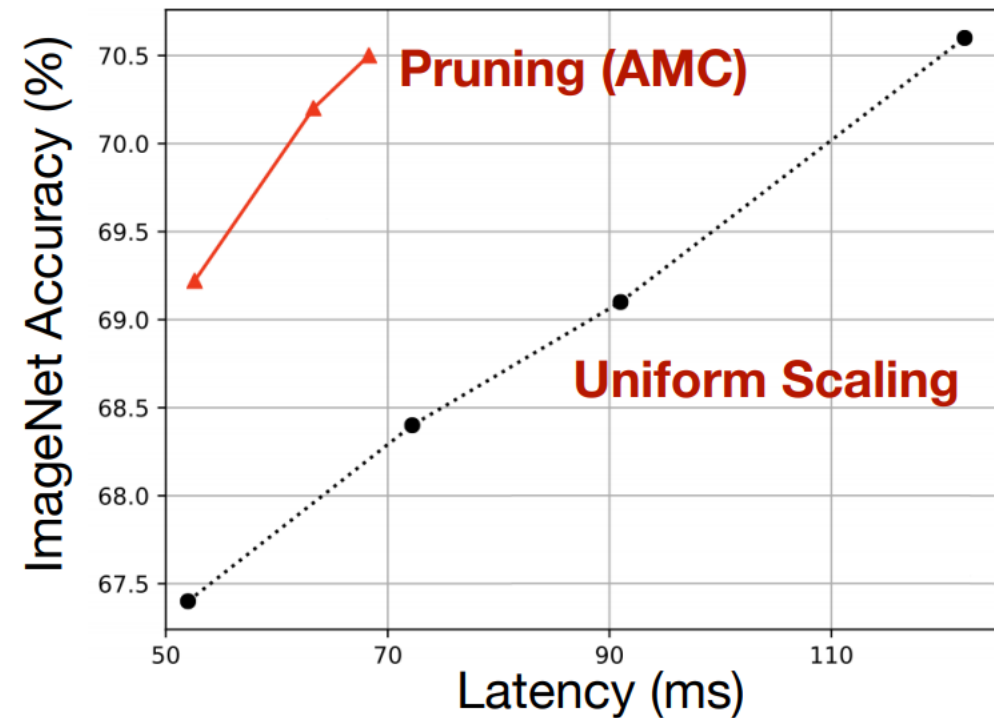
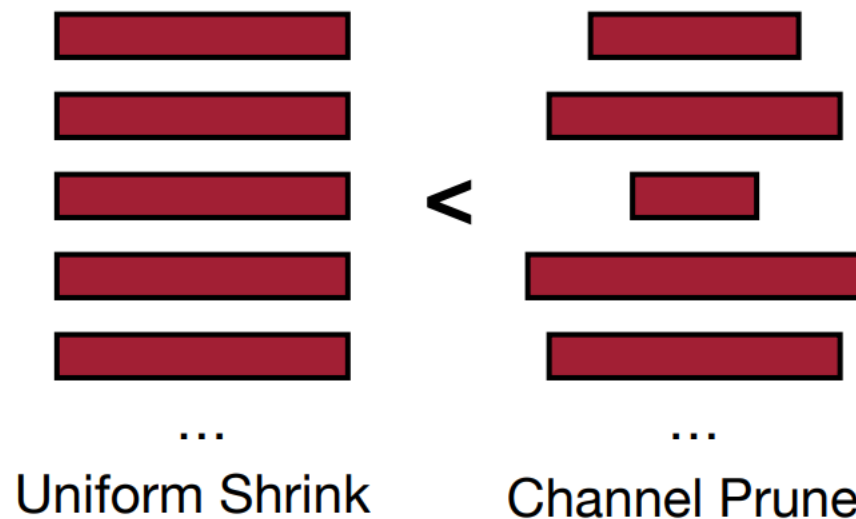
(A100 can support)



Layer-wise N:4

Layer-wise Pruning (cont.)

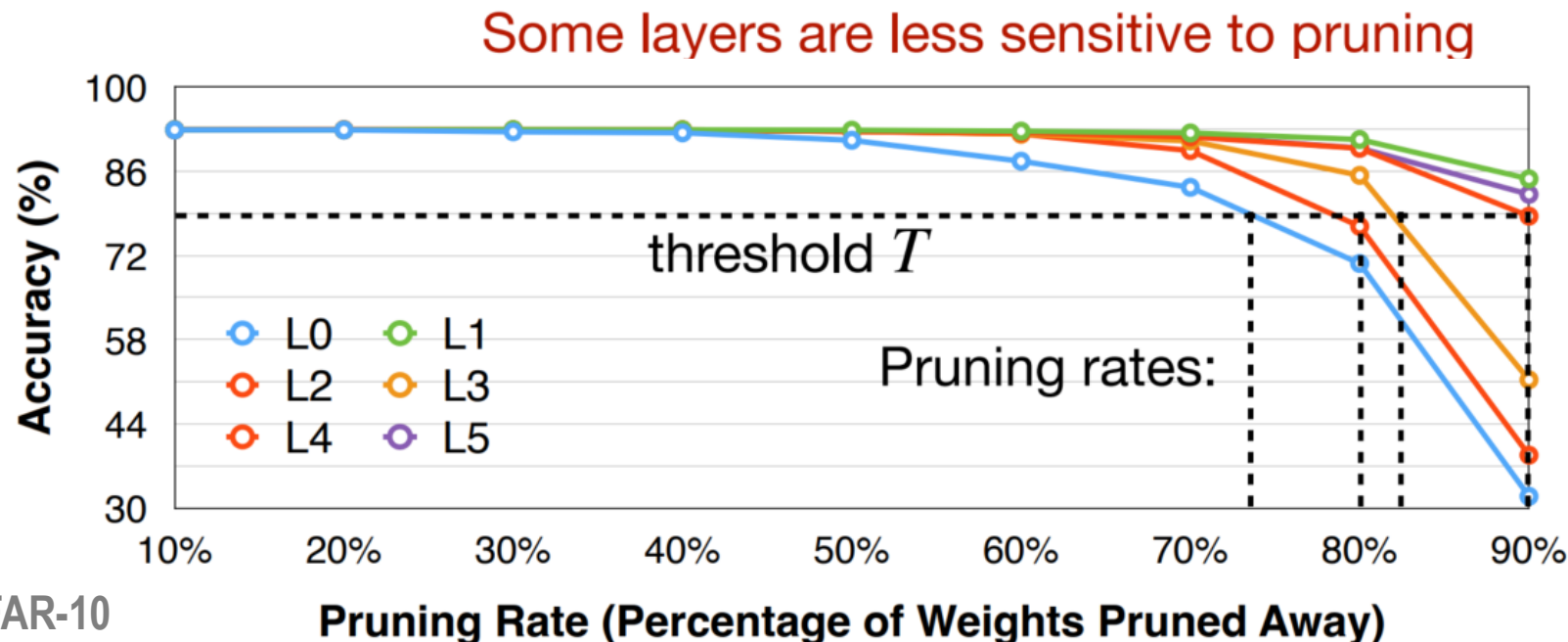
- Non-uniform pruning is better than uniform shrinking



Question: how should we find ratios for each layer?

Per-layer Sensitivity Analysis

1. Pick a layer L_i in the model
2. Prune the layer L_i with pruning ratio $r \in \{0, 0.1, 0.2, \dots, 0.9\}$ (or other strides)
3. Observe the accuracy degrade ΔAcc_r^i for each pruning ratio
4. Repeat the process for all layers
5. Pick a recommendation threshold T such that the accuracy degrade is desired



Is this optimal?
→ maybe not...

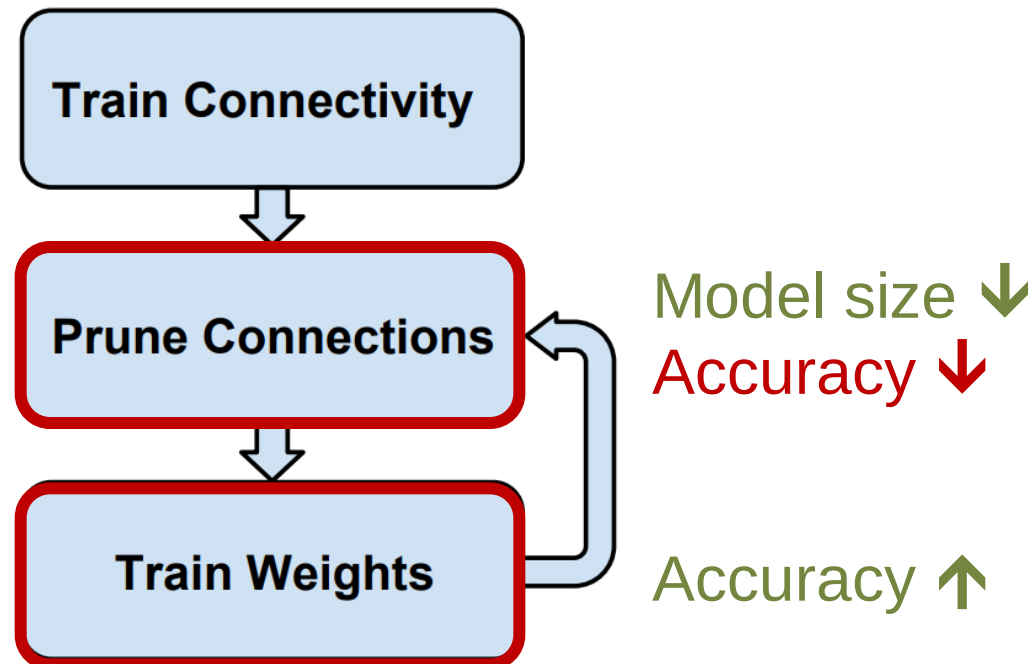
Outline

- Motivation of pruning
- Concept of pruning
- Introduction of different pruning methods
 - Unstructured pruning
 - Structured pruning
 - Semi-structured pruning
- Hardware for pruned/sparse NNs
- **Fine-tuning/training pruned neural networks**



Fine-Tune a Pruned Neural Network

- To reduce the impact of pruning on accuracy
- Other flows will be introduced in a latter chapter (NAS)

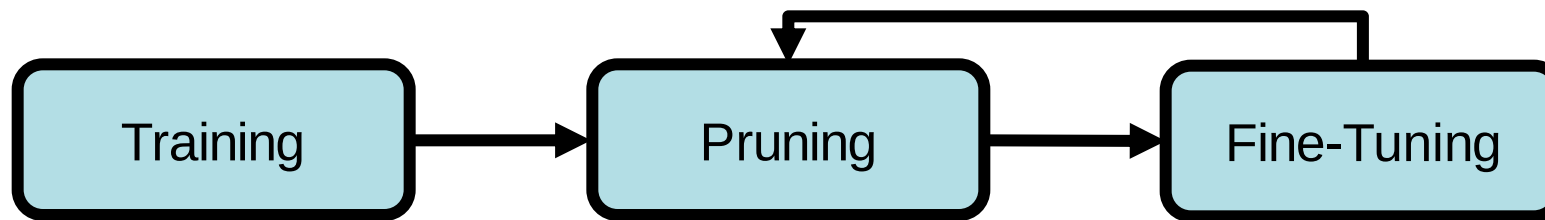


One-Shot Pruning & Iterative Pruning

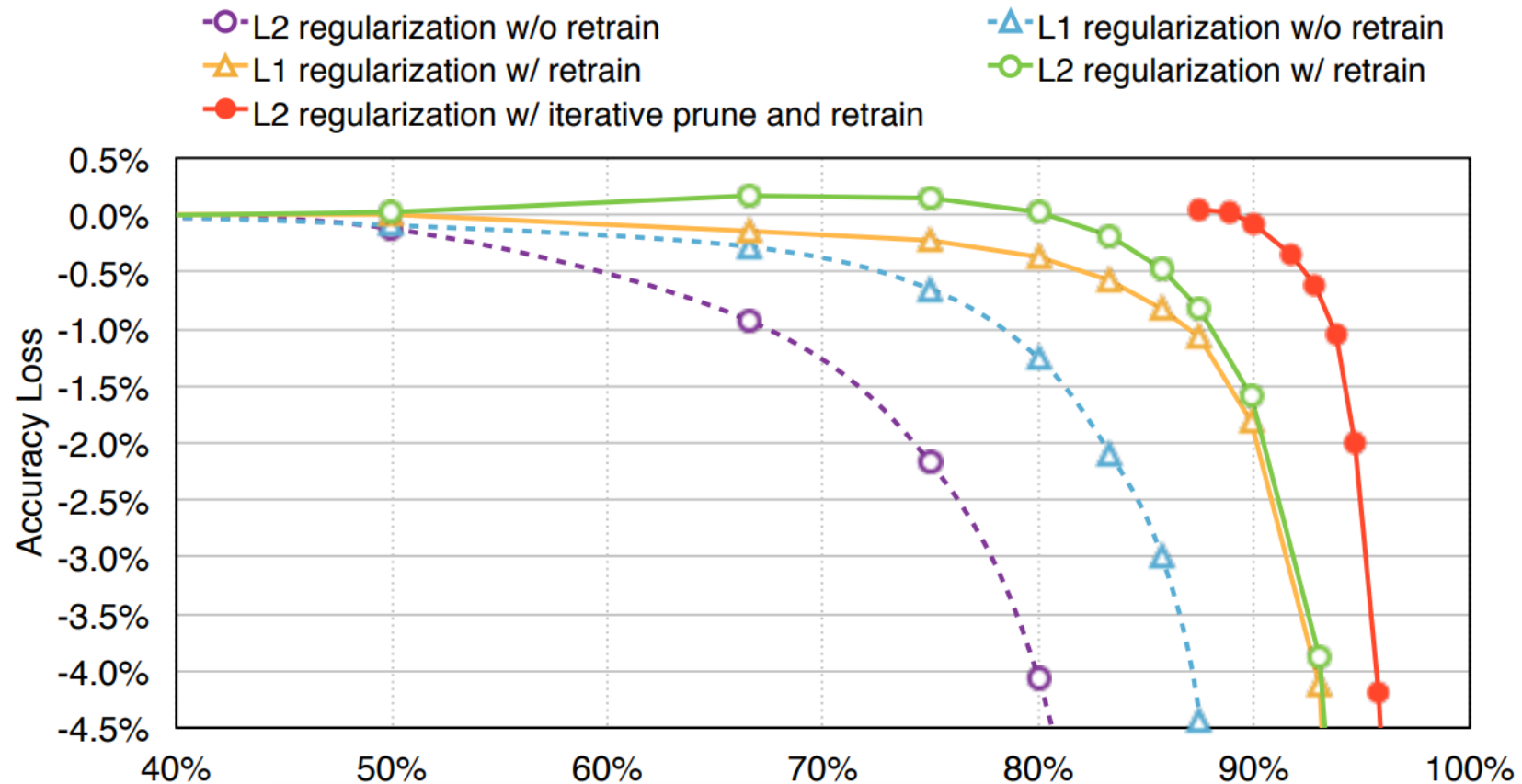
■ One-shot pruning



■ Iterative pruning



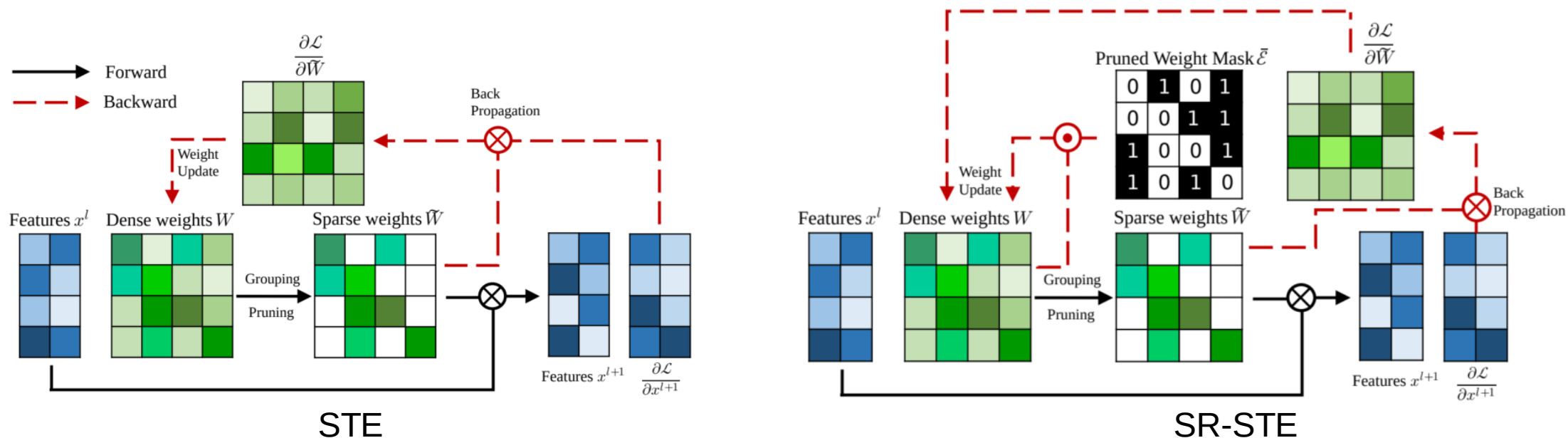
Effect of Fine-Tuning



SR-STE (Zhou et al., ICLR 2021)

■ Sparse-refined straight-through estimator (SR-STE)

- Update both to-be-pruned and remaining weights according to mask



$$\mathcal{W}_{t+1} \leftarrow \mathcal{W}_t - \gamma_t g(\tilde{\mathcal{W}}_t)$$

$$\tilde{\mathcal{W}}_j = \mathcal{W}_j \odot \mathcal{E}_j$$

$$\mathcal{W}_{t+1} = \mathcal{W}_t - \gamma_t (g(\tilde{\mathcal{W}}_t) + \lambda_W (\bar{\mathcal{E}}_t \odot \mathcal{W}_t))$$



Why Pruning?

■ How about simply train a smaller network?

- It is widely known that smaller network is more difficult to learn successfully ☹
- Larger network is easier to optimize?

■ Lottery Ticket Hypothesis (LTH)

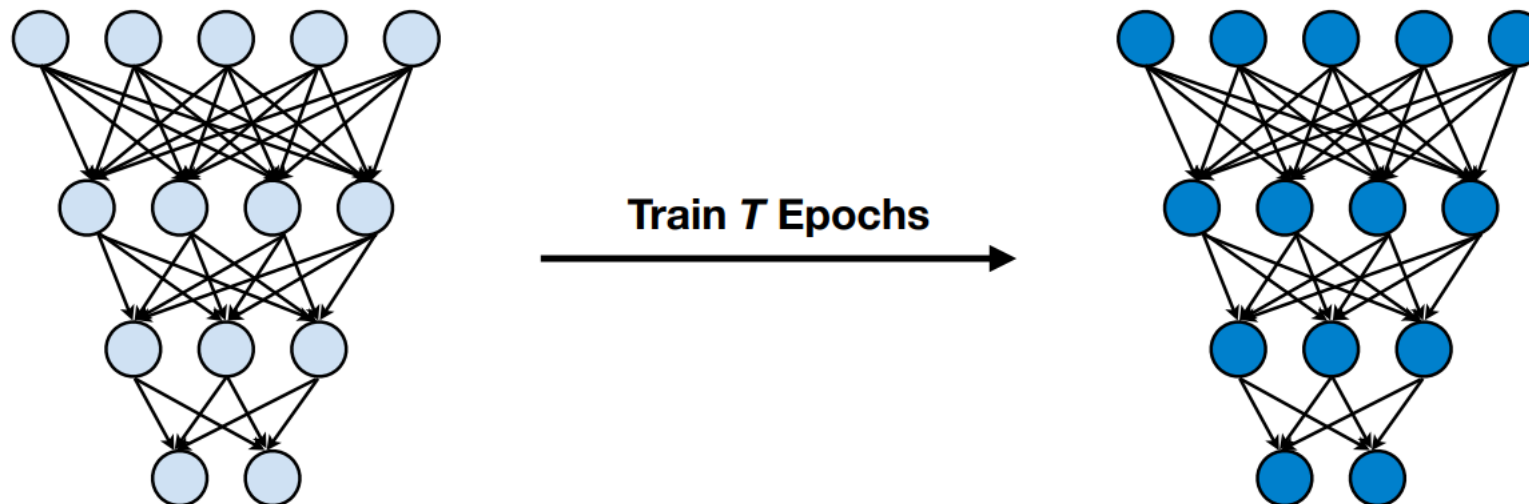
- J. Frankle *et al.*, *ICLR* 2019



Lottery Ticket Hypothesis (J. Frankle et al., ICLR 2019)

■ The Lottery Ticket Hypothesis

- *A randomly-initialized, dense neural network contains a subnetwork that is initialized such that—when trained in isolation—it can match the test accuracy of the original network after training for at most the same number of iterations*



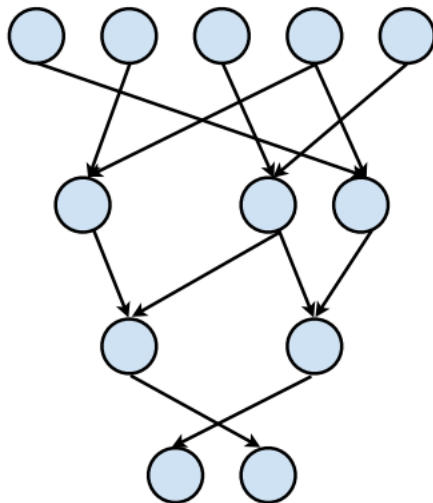
Randomly-Initialized

Lottery Ticket Hypothesis (J. Frankle et al., ICLR 2019)

■ The Lottery Ticket Hypothesis

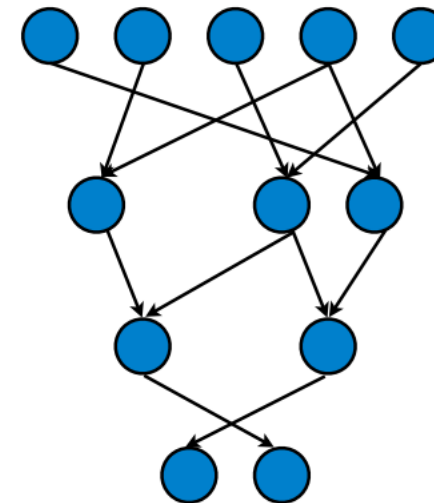
- A randomly-initialized, dense neural network contains a subnetwork that is initialized such that—when trained in isolation—it can match the test accuracy of the original network after training for at most the same number of iterations.

Winning Ticket



Initialized Subnetwork

Train at most T Epochs



Accuracy Matched

Find the Winning Ticket

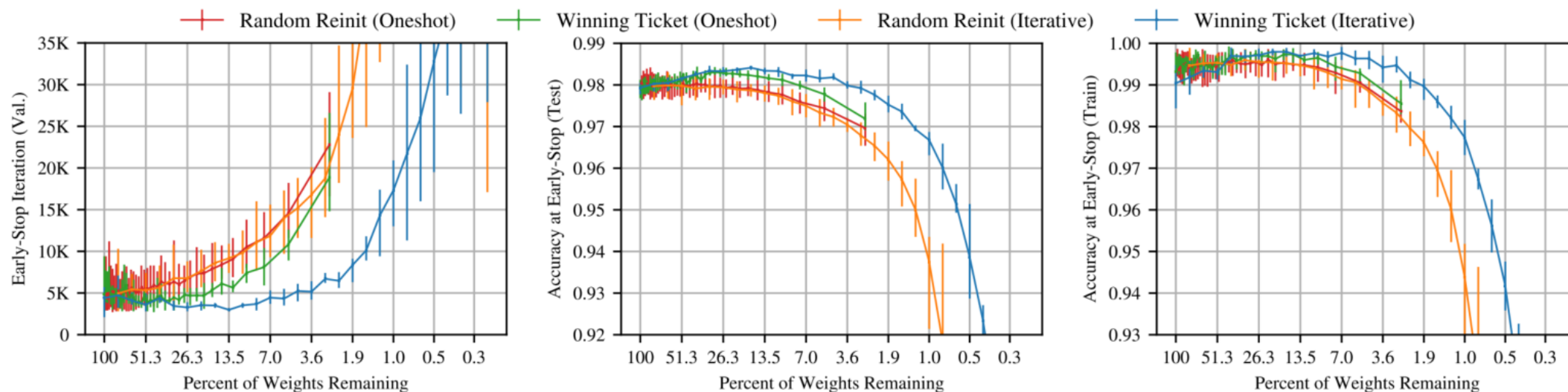
1. Randomly initialize a neural network $f(x; \theta_0)$ (where $\theta_0 \sim \mathcal{D}_\theta$).
2. Train the network for j iterations, arriving at parameters θ_j .
3. Prune $p\%$ of the parameters in θ_j , creating a mask m .
4. Reset the remaining parameters to their values in θ_0 , creating the winning ticket $f(x; m \odot \theta_0)$.



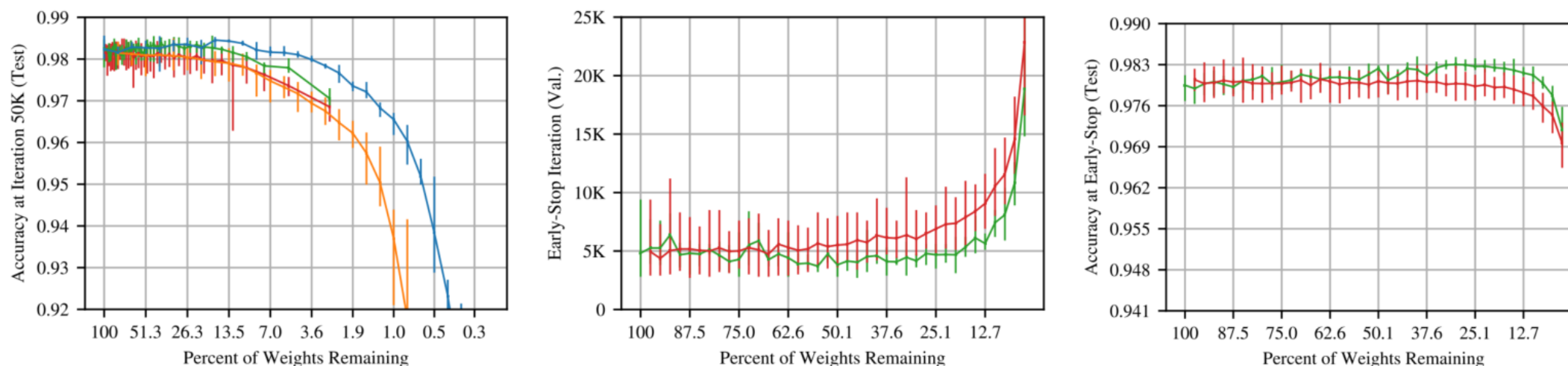
Experimental Results from LTH

Dataset: MNIST

NN: Lenet-300-100 (LeCun *et al.*, 1998)



(a) Early-stopping iteration and accuracy for all pruning methods.



(b) Accuracy at end of training.

(c) Early-stopping iteration and accuracy for one-shot pruning.

J. Frankle *et al.*, "The lottery ticket hypothesis: finding sparse, trainable neural networks," in *Proc of ICLR*, 2019.

After Lottery Ticket Hypothesis

- **Deconstructing lottery tickets: zeros, signs, and the supermask**
 - H. Zhou *et al.*, *NeurIPS* 2019
 - 3 components to find the winning tickets
- **Rethinking the value of network pruning**
 - Z. Liu *et al.*, *ICLR* 2019
 - LTH successes only when small network, small dataset, and small lr



Summary of this Chapter

- To deploy high-accuracy NNs on edge devices, we need smaller models!
- Pruning methods
 - Unstructured pruning, structured pruning, & semi-structured pruning
 - Saliency score of a weight (or a group of weights)
- Hardware for pruned/sparse NNs
- Pruning + fine-tuning

